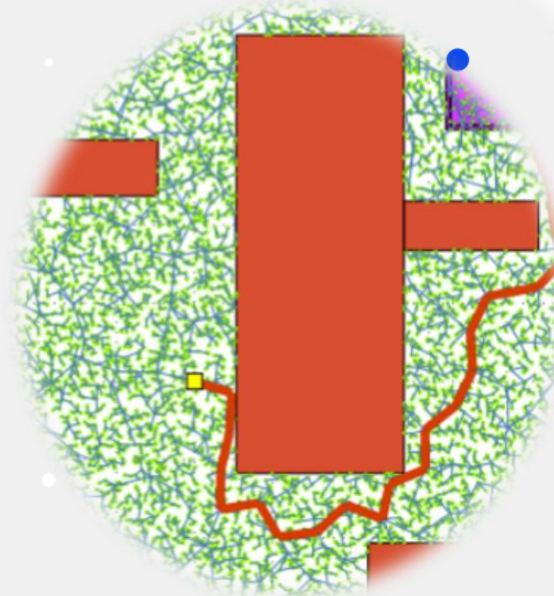
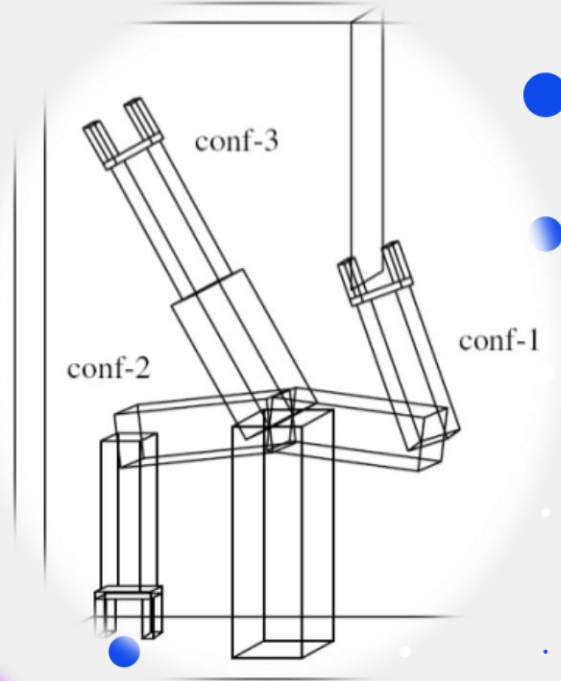
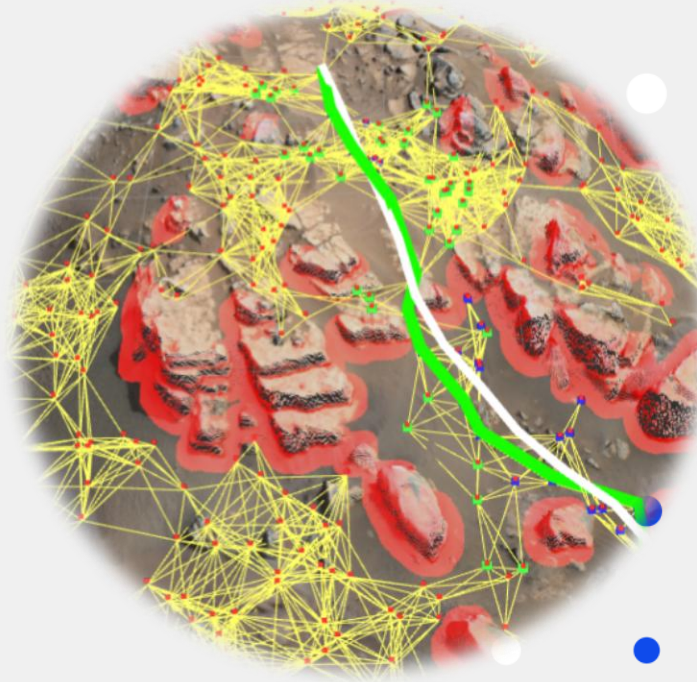


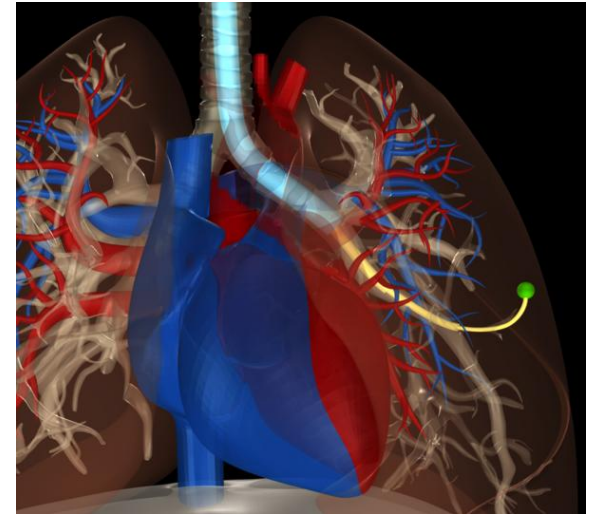
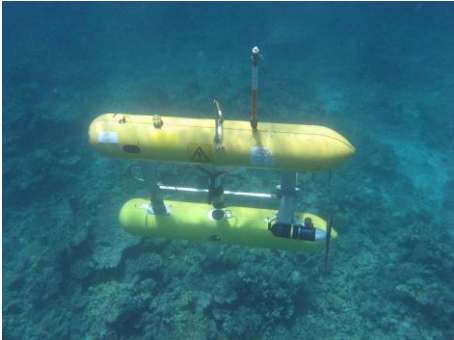


Path Planning

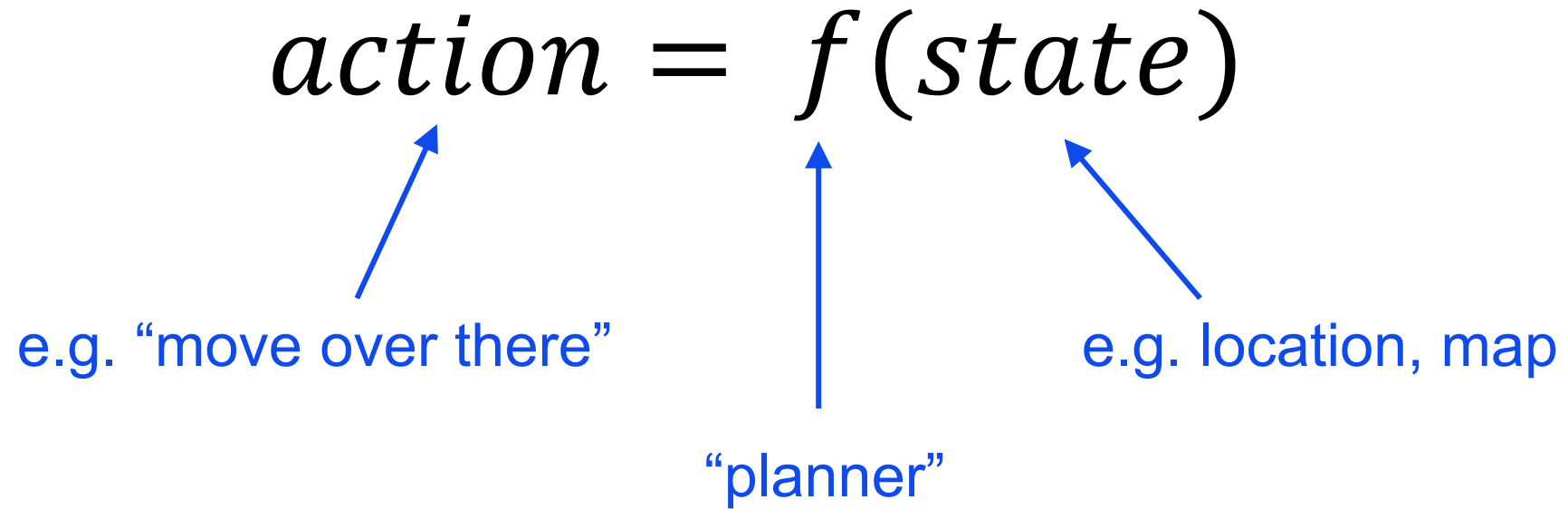
Graeme Best
41068 Robotics Studio 1

Faculty of Engineering and Information Technology
University of Technology Sydney

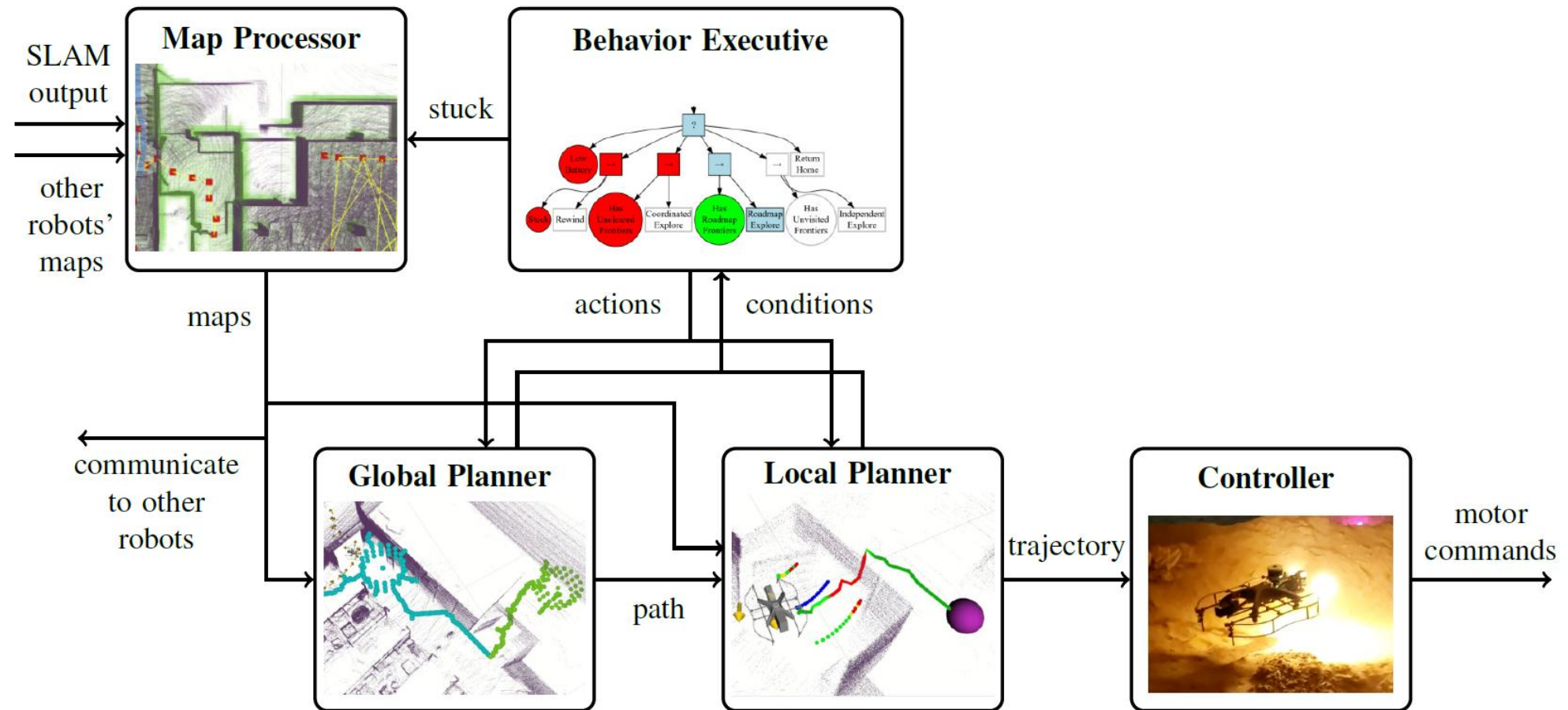




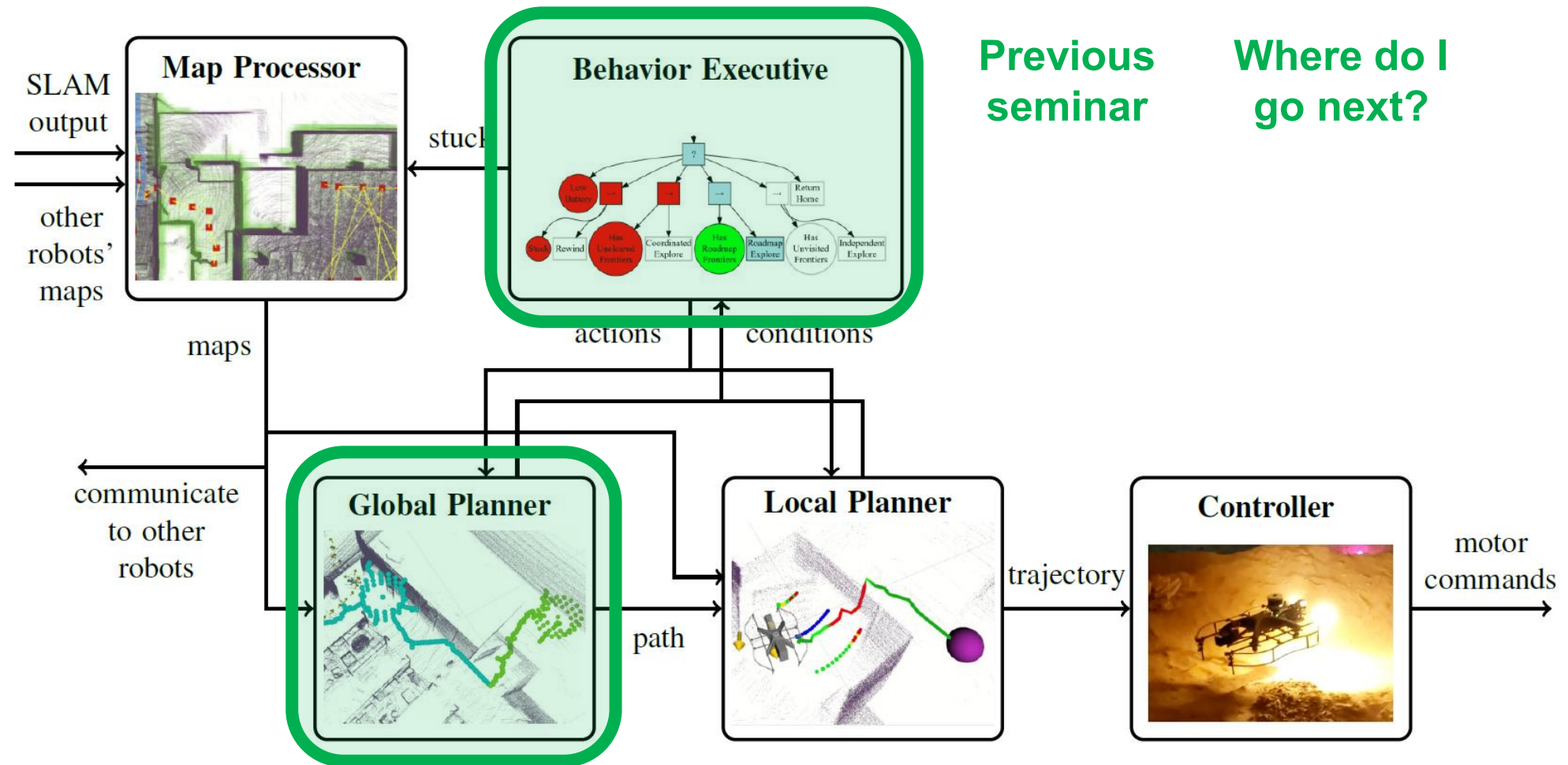
Robot Decision Making



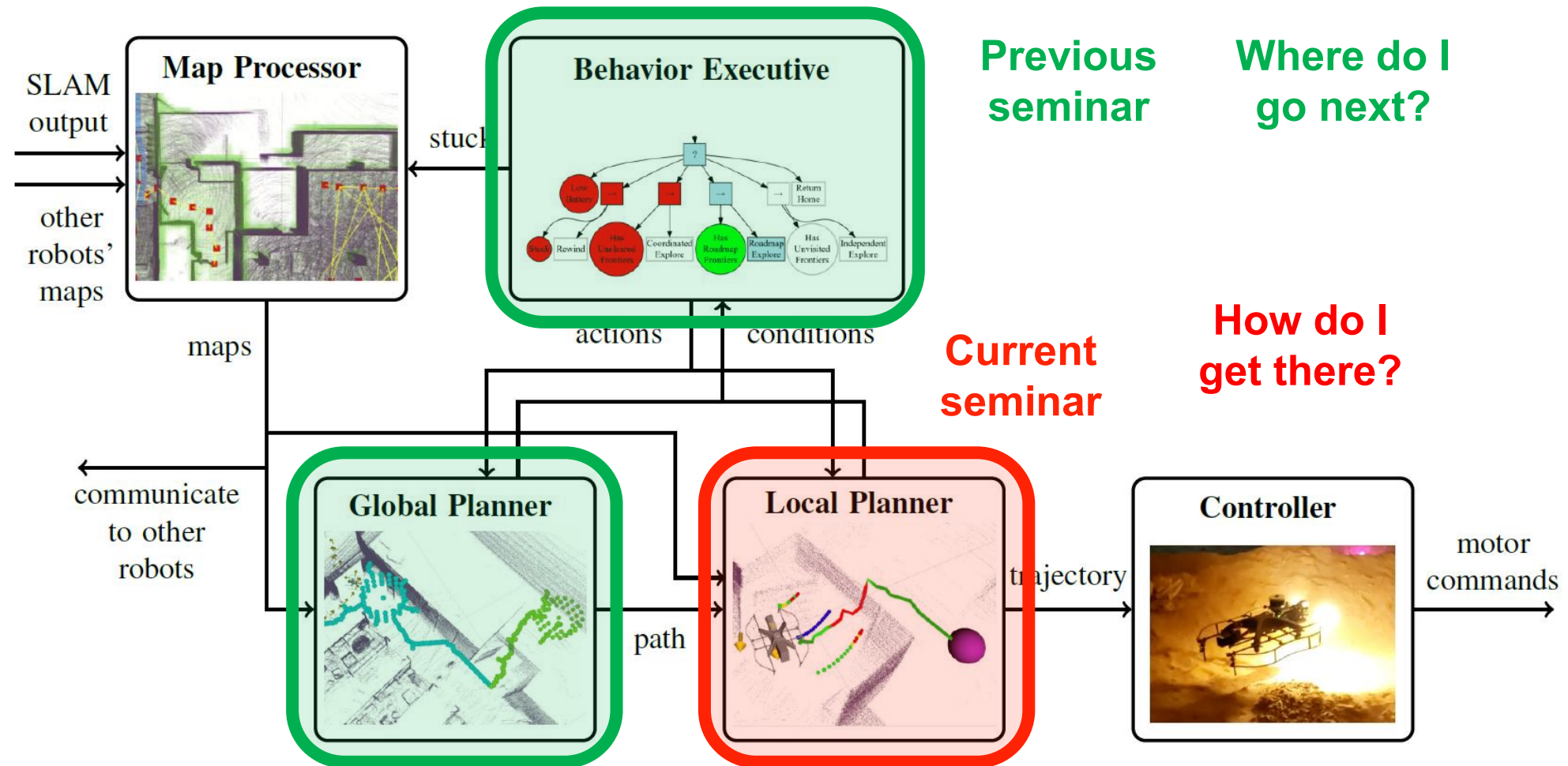
Example Decision Making Hierarchy



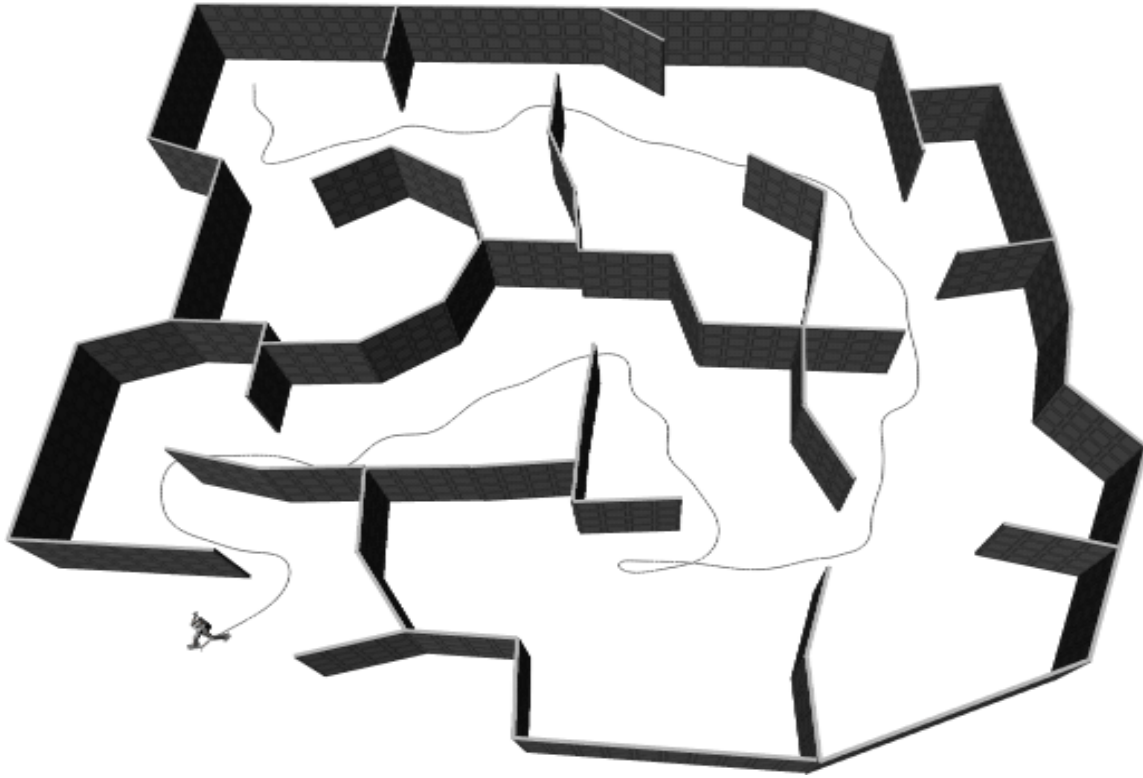
Example Decision Making Hierarchy



Example Decision Making Hierarchy



Definition: Path Planning



Path Planning:

- Given the map, the current robot location, and the goal
- Find a trajectory that will cause the robot to reach the goal location when executed

What is Path Planning?



- **Robot path planning** is the process of figuring out *how the robot moves from its current state to a target* given its map, constraints, uncertainties.
- It includes computing feasible, safe paths/trajectories; handling obstacles and dynamic objects; respecting kinematic/dynamic limits and non-holonomic constraints; and replanning as the world changes
- Good path planning enables smooth, efficient, and safe motion, making real-time progress toward goals while balancing time, energy, and risk.

Why is Path Planning Challenging?



- Defining the right objective: time, speed, safety, comfort
- Incomplete and uncertain information – usually from noisy sensors
- Interact with high-level decision making
- Interact with lower-level controllers
- Computationally difficult – many paths to pick from
- Choosing the right algorithm for the task
- Tuning algorithm parameters
- Multi-robot collision avoidance

Questions for your 41068 Project



- Introduce standard path planning techniques
- Your project must have some decision making / path planning
- How will your robot navigate around the environment?
 - You are probably using the Nav2 stack
 - But how does this work?
 - How can you modify it?
 - Or implement it yourself?



Outline



- Quick definitions
- Artificial potential fields
- Graph search methods
- Sampling-based motion planning
- Nav2 navigation stack



Quick Definitions

Configuration Space

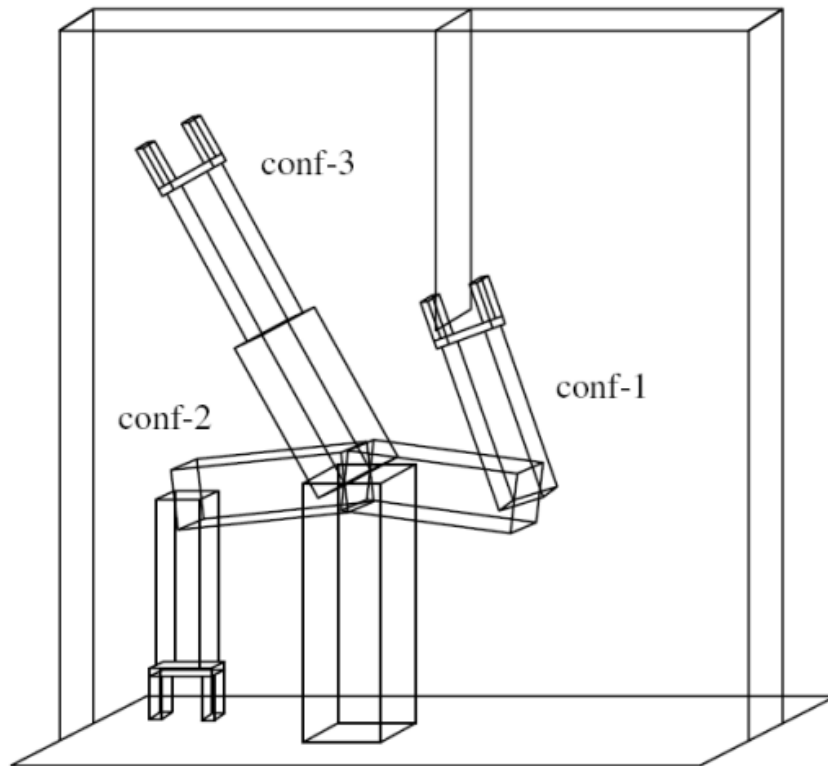


- Although the motion planning problem is defined in the regular world (the “workspace”), it lives in another space: **the configuration space**
- A robot configuration q is a specification of the position of all robot points relative to a fixed coordinate system
- Usually a configuration is expressed as a **vector of positions and orientations**
- The configuration space (“C-space”) is the **space of all possible configurations**
- The topology of C-space is usually **not the Cartesian space**

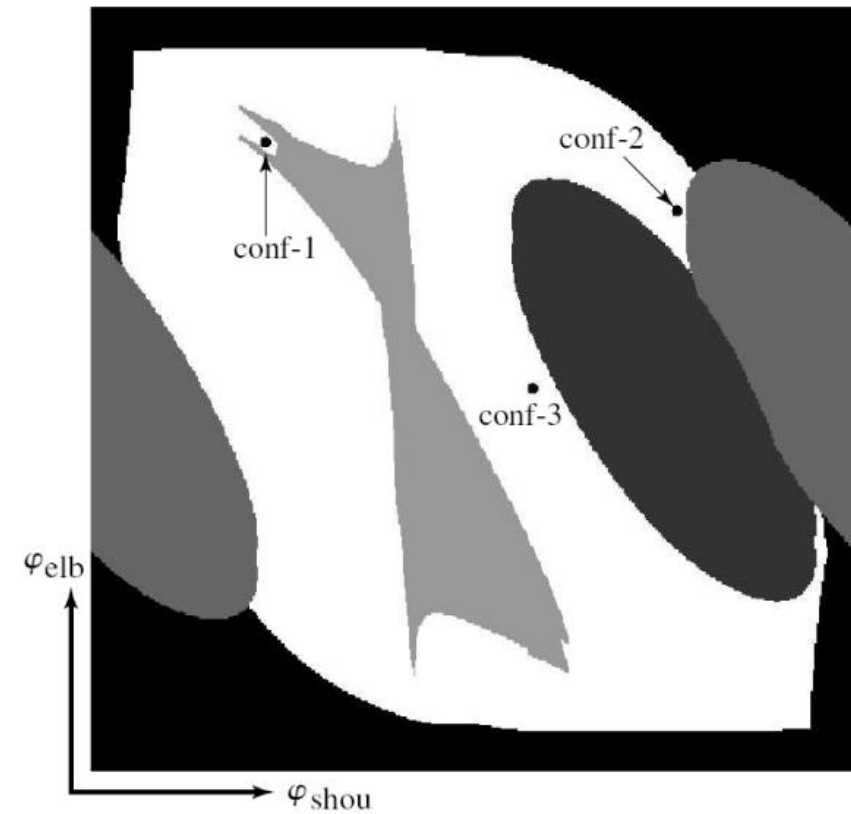
Example: Robot Arms



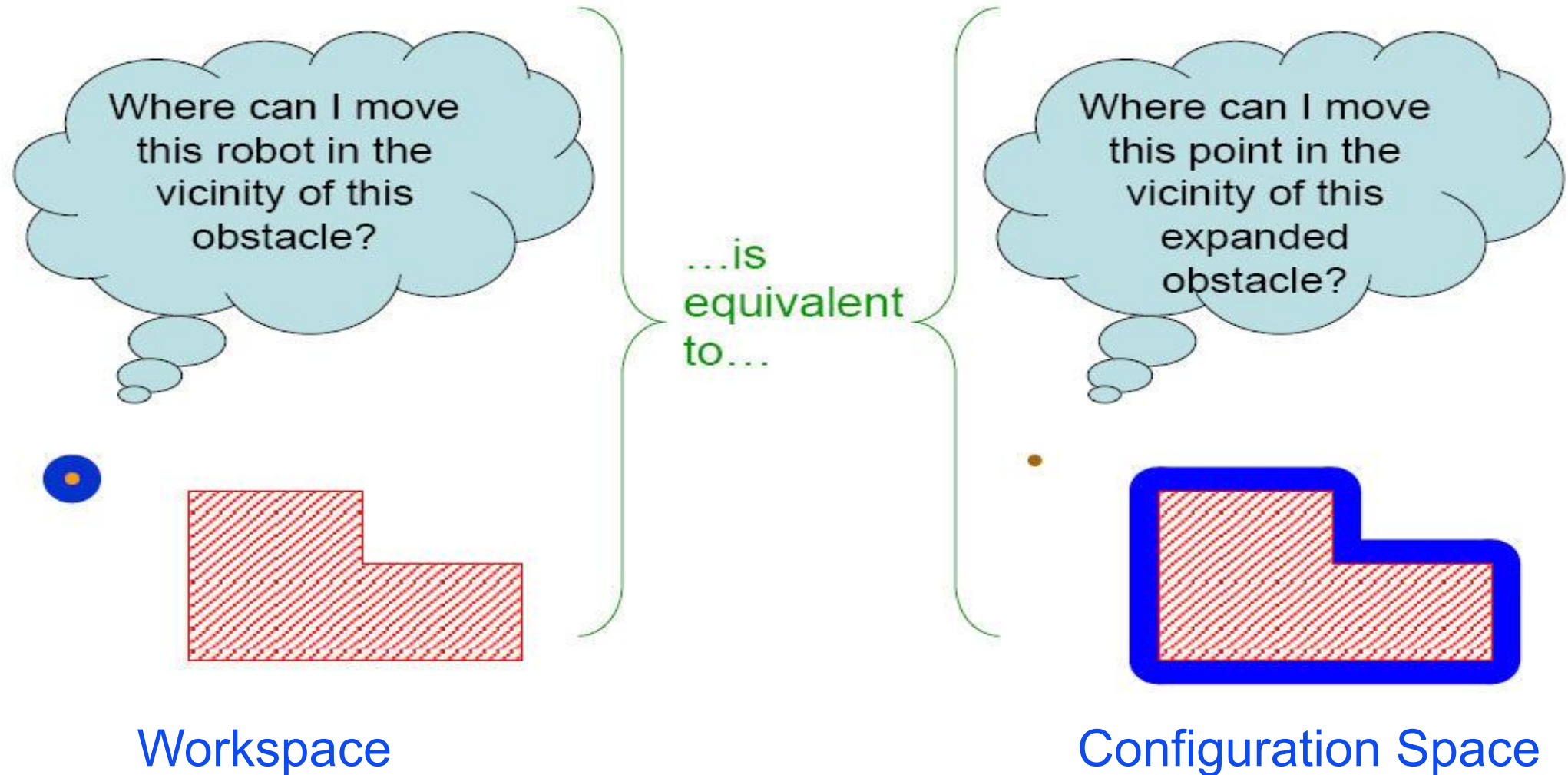
3 different configurations



Configuration space



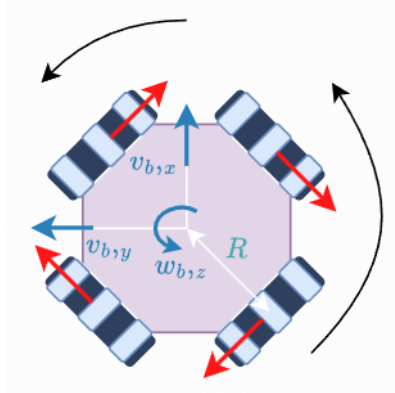
Example: 2D World, 2 Degrees of Freedom, Circle



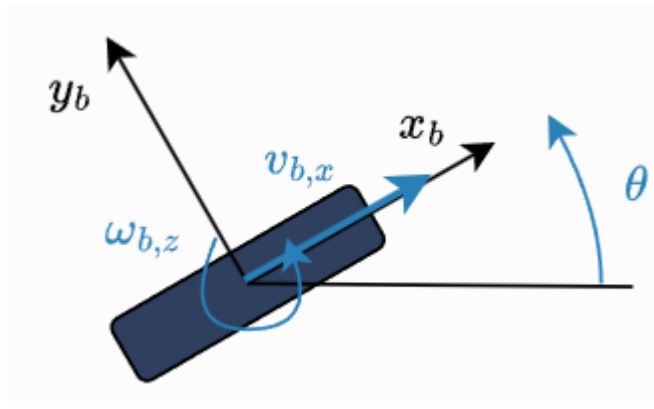
Motion Models



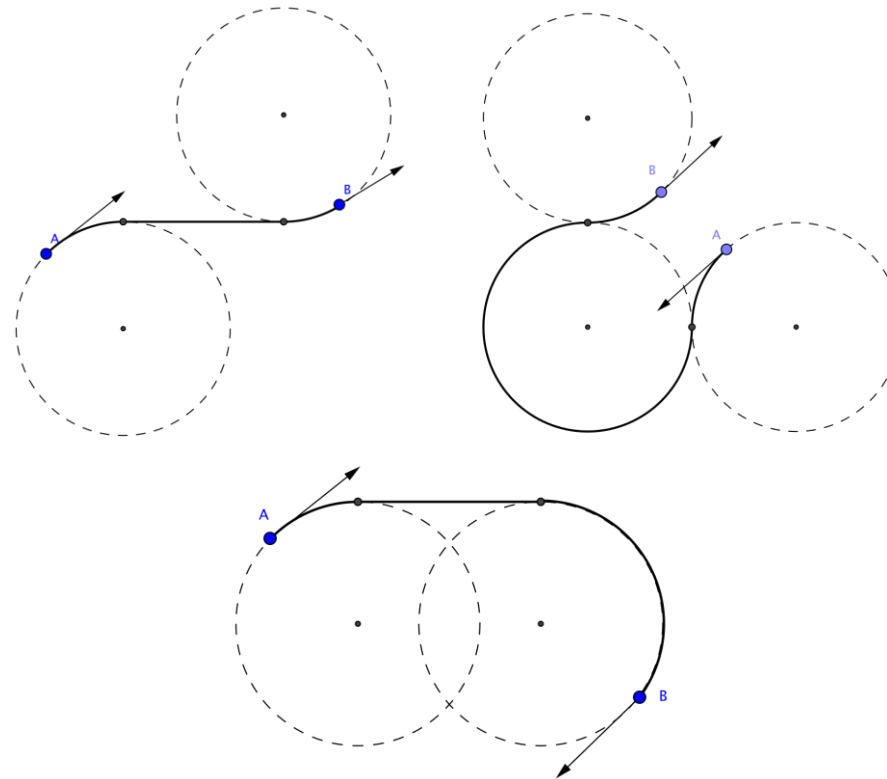
Omnidirectional



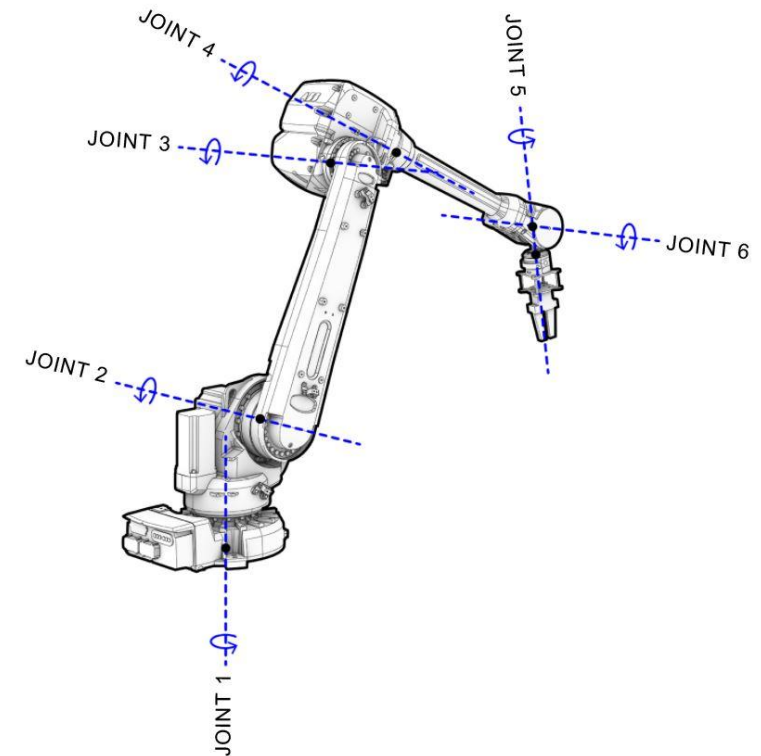
Unicycle



Dubins car



Industrial robot arm

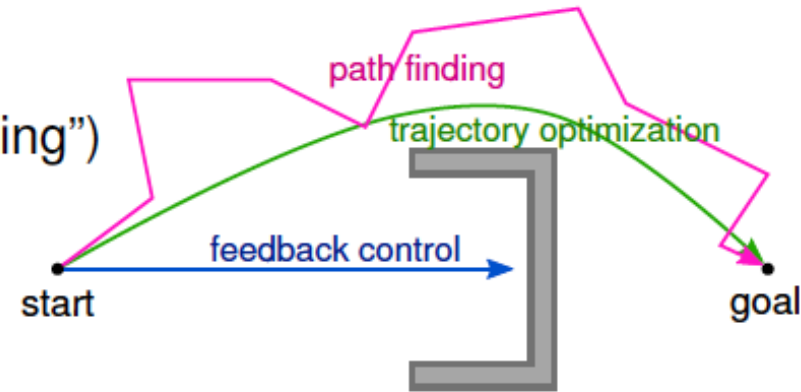


Method Variations



Control, path finding, trajectory optimization

- Combining methods:
 - 1) Path Finding
 - 2) Trajectory Optimization (“smoothing”)
 - 3) Feedback Control



- Many problems can be solved with only feedback control (though not optimally)
- Many more problems can be solved *locally* optimal with only trajectory optimization
- Tricky problems need path finding: *global* search for valid paths



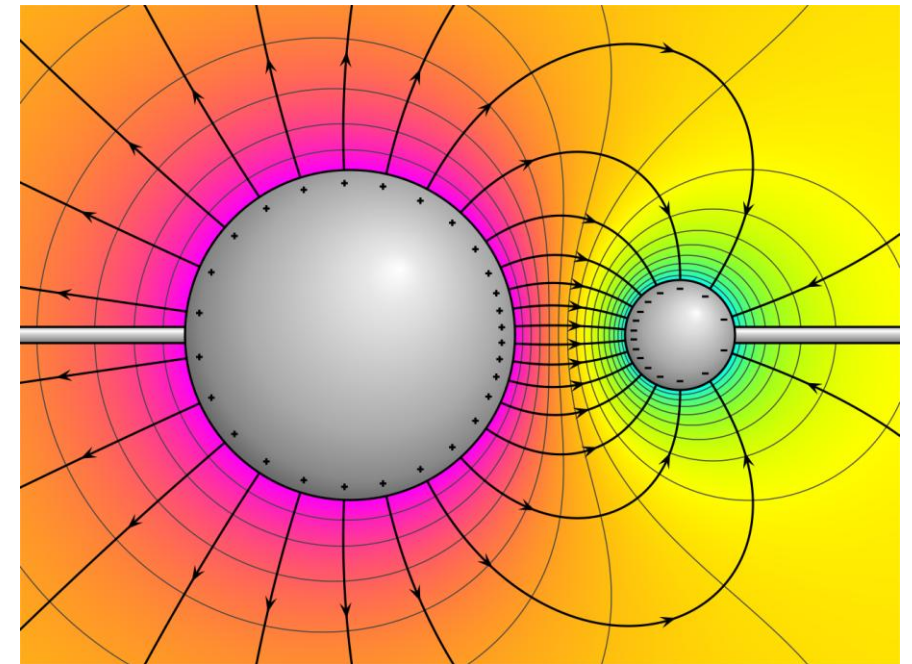
Artificial Potential Fields

Artificial Potential Fields

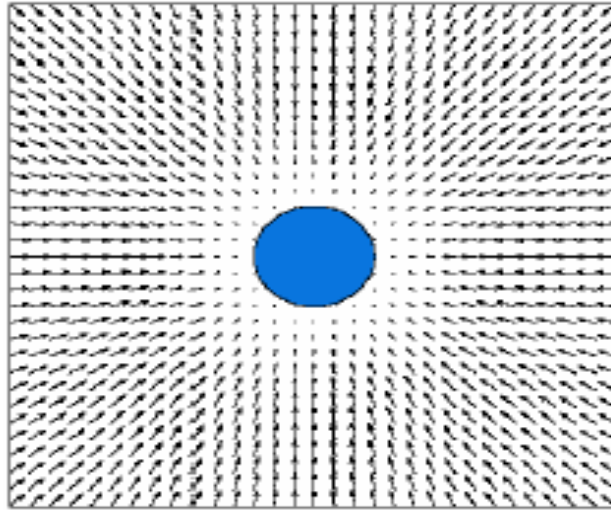


- Attempts to guide the robot from the initial configuration to the goal configuration while staying in C-free
- Analogous to an electric potential field
- Potential field composed of:
 - Attraction force at goal
 - Repulsion force at obstacles
- The robot follows a virtual force defined by the gradient of the potential field

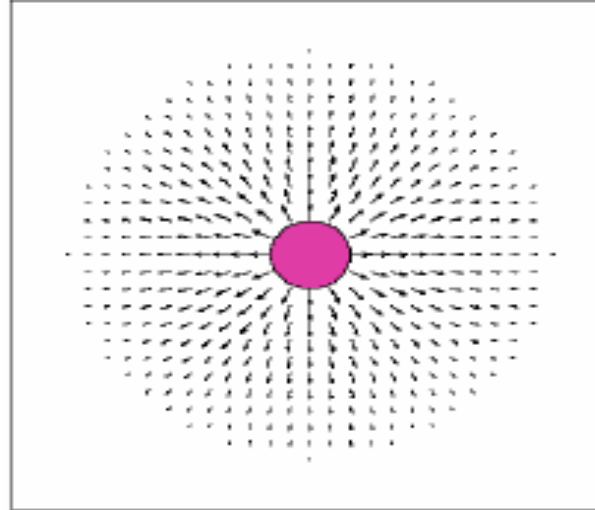
Electric potential field



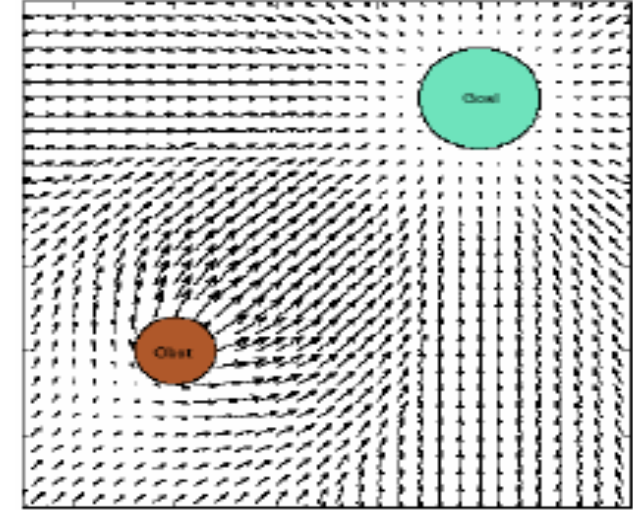
Example



Attractive Potential
for goals



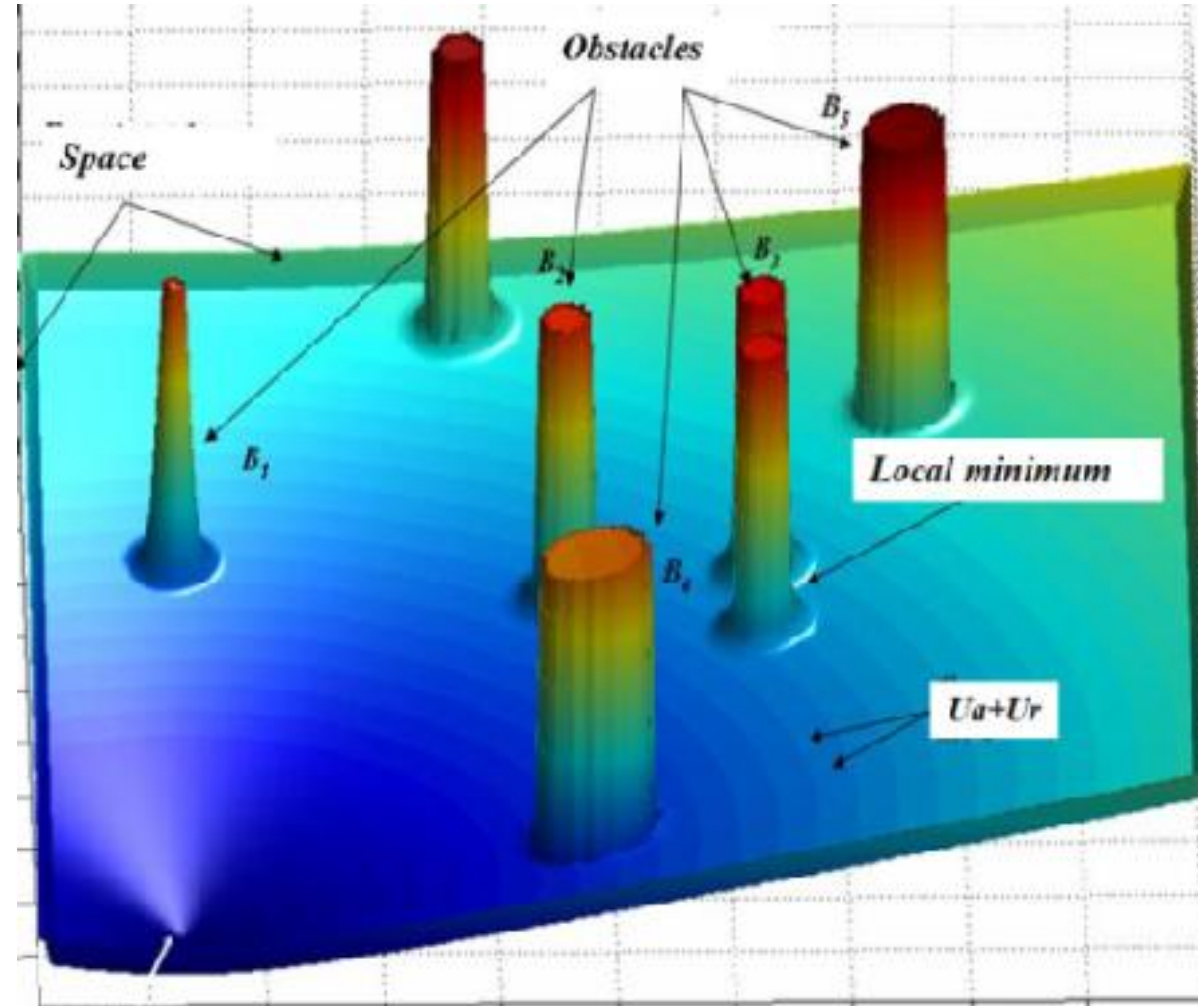
Repulsive Potential
for obstacles



Combined Potential
Field

Move along force: $F(x) = \nabla U_{\text{att}}(x) - \nabla U_{\text{rep}}(x)$

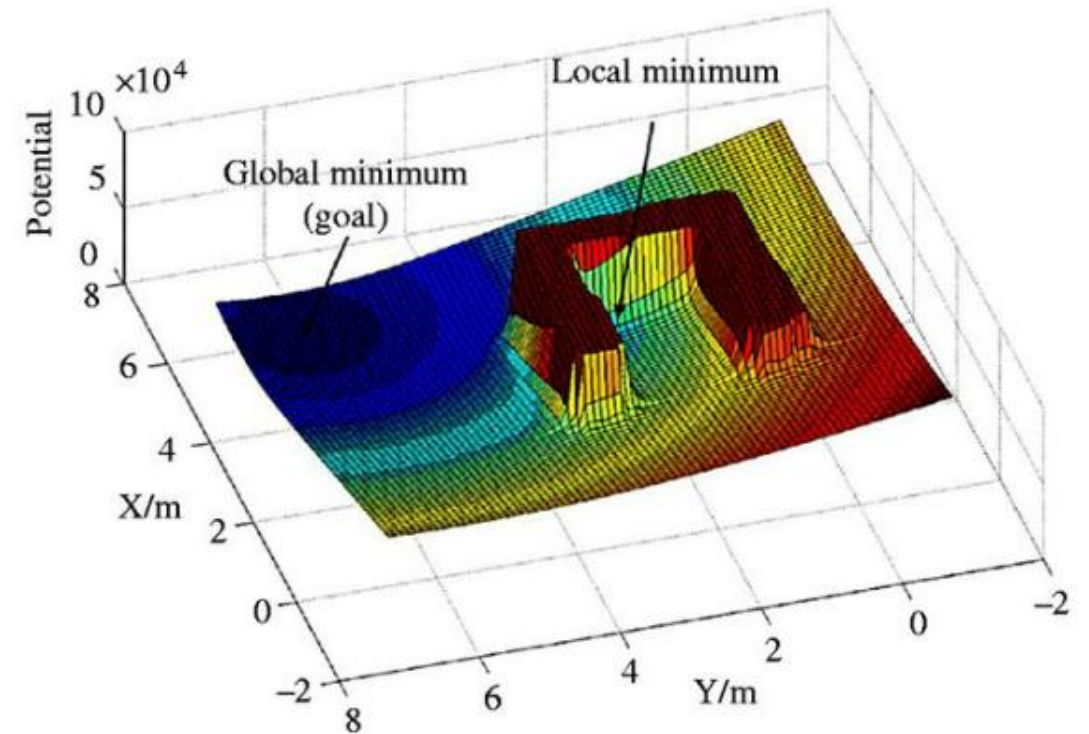
Many Obstacles: Move Downhill



When does it go wrong?



- Stuck in local minima
- How to fix this?
 - Random walk when stuck
 - Define potential fields without local minima (?)
 - Combine with global path planning





Graph Search

Graph Search Method



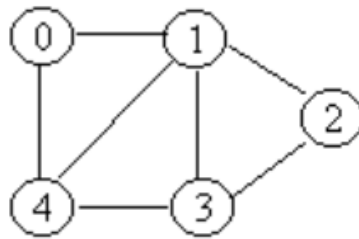
1. Discretise C-Space into a graph
 - Grid cells
 - Visibility graph
 - Probabilistic roadmap
2. Perform a graph search
3. Extract the shortest path

Graph Data Structure



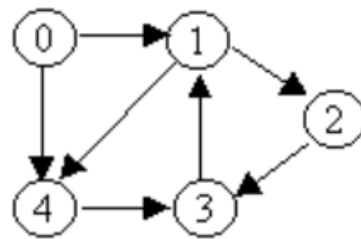
- Graph $G = (V, E)$ where V is a set of **vertices** and E is a set of **edges**.

Each edge $e \in E$ is a 2-tuple of the form (v, w) where $v, w \in V$, and e is called an **incident** on v and w .

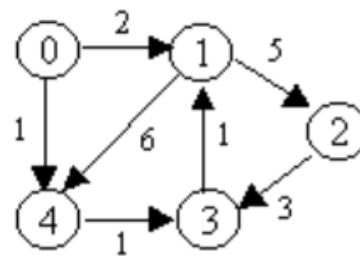


$V = \{0, 1, 2, 3, 4\}$
 $E = \{(0, 1), (1, 2), (1, 3), (1, 4),$
 $(2, 3), (3, 4), (4, 0)\}$

- An edge may be directed or undirected.
- An edge may also have a **weight**.



Directed, unweighted

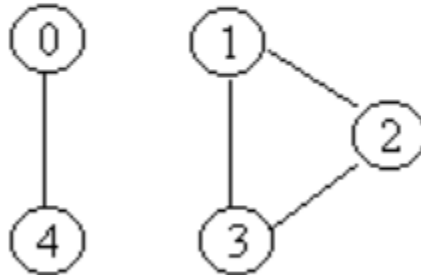


Directed, weighted

Graph Data Structure



- A **path** is a sequence of vertices connected by edges, and represented as a sequence in 2 ways:
 - $(v_0, e_1, v_1, e_2, v_2, \dots, v_{n-1}, e_n, v_n)$ -- alternating vertices and edges
 - $(v_0, v_1, v_2, \dots, v_{n-1}, v_n)$ -- vertices only
- A graph is **connected** if, for any vertices v and w , there is a path from v to w .



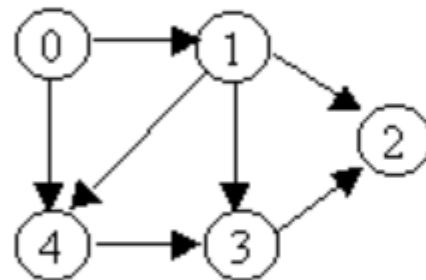
An unconnected graph

Graph Data Structure

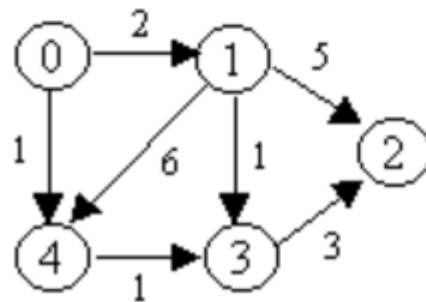


2. Graph Representations

- Adjacency matrix
 - n by n matrix, where n is number of vertices
 - $A(i,j) = 1$ iff (i,j) is an edge, or 0 otherwise
 - For weighted graph: $A(i,j) = w$ (weight of edge), or positive infinity otherwise



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

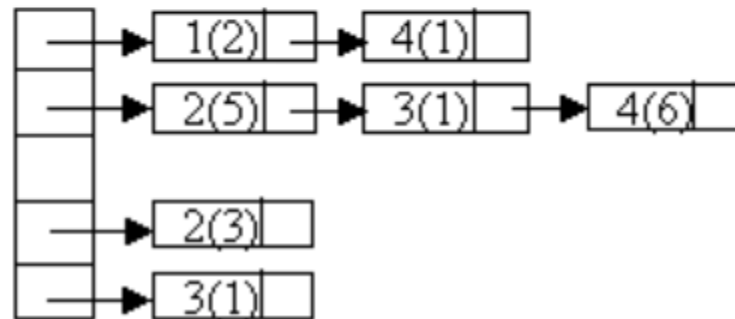
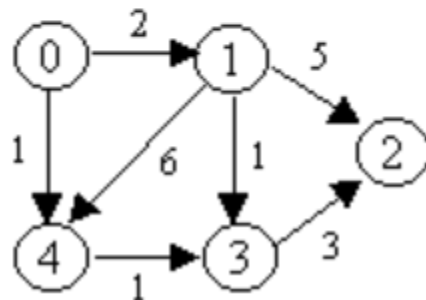
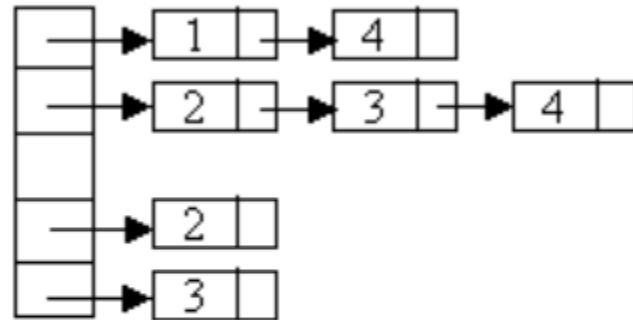
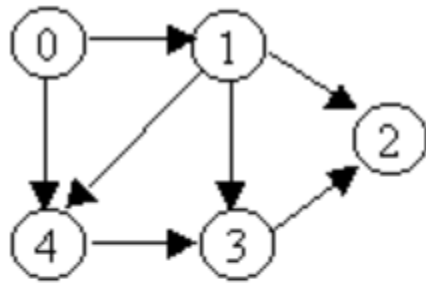


$$A = \begin{bmatrix} \infty & 2 & \infty & \infty & 1 \\ \infty & \infty & 5 & 1 & 6 \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 3 & \infty & \infty \\ \infty & \infty & \infty & 1 & \infty \end{bmatrix}$$

Graph Data Structure



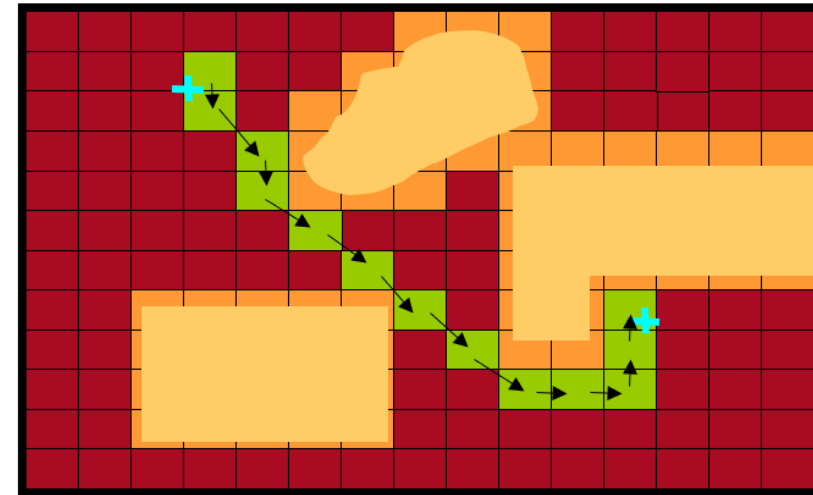
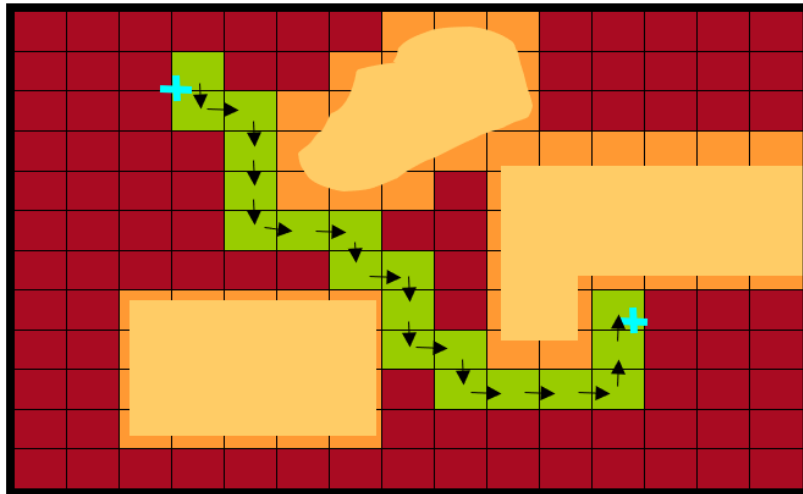
- Adjacency list
 - Each vertex has a linked list of edges
 - Edge stores destination and label
 - Better when adjacency matrix is **sparse**



C-free Space Discretisation: Cell Decomposition



1. Create a grid
2. Avoid any cells that intersect with an obstacle
3. Connect a graph over the cells
4. Plan shortest path
5. If no plan exists, double the resolution and try again!



Wavefront Algorithm



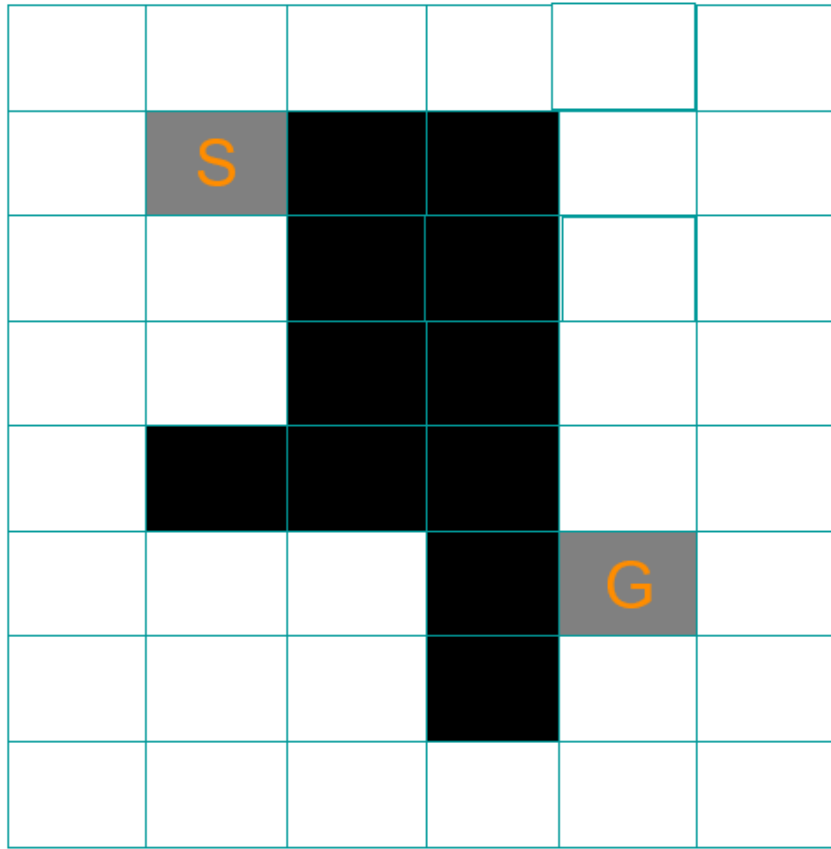
- Motivated by water waves
 - 1. Create a discretised map using cell decomposition
 - 2. Add in start and goal locations
 - 3. Fill in the “wavefront table”
 - 4. Get the obstacle-free path from the wavefront table
- (Many ways to implement this)



Wavefront Algorithm



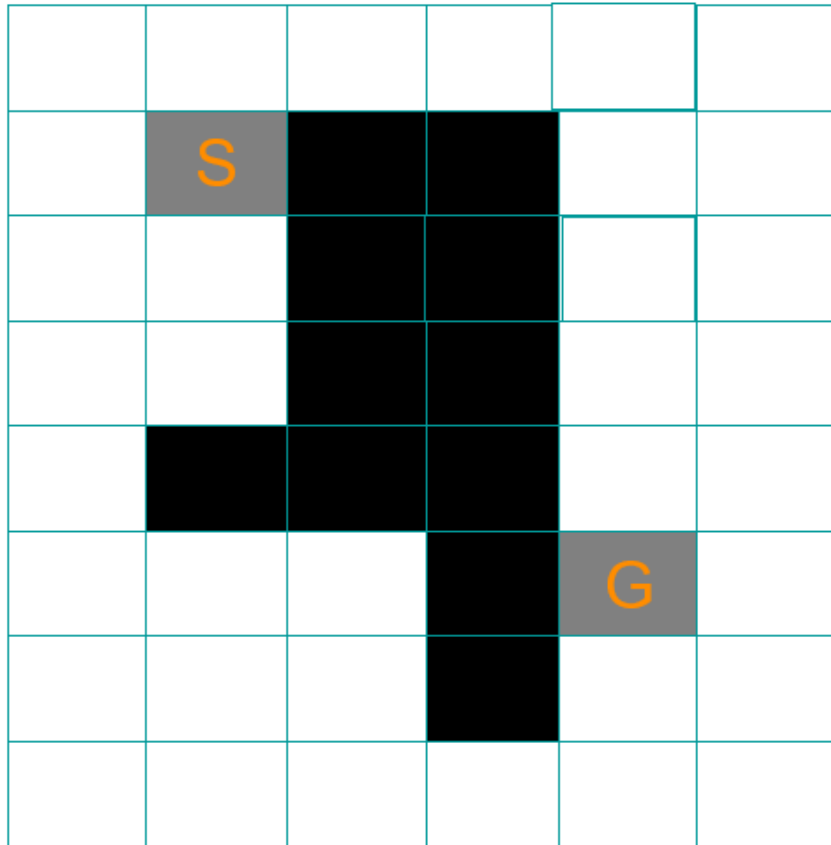
S– start, G--goal



Wavefront Algorithm



S– start, G--goal



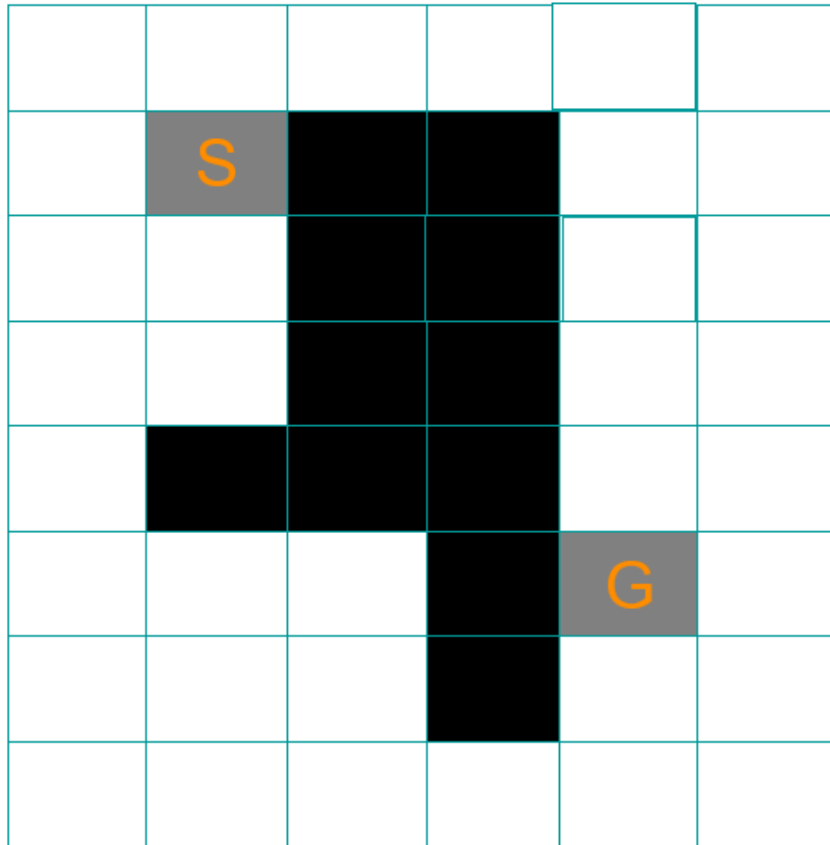
Build the wave (start from goal)



Wavefront Algorithm



S– start, G--goal



Get the path (reverse order)

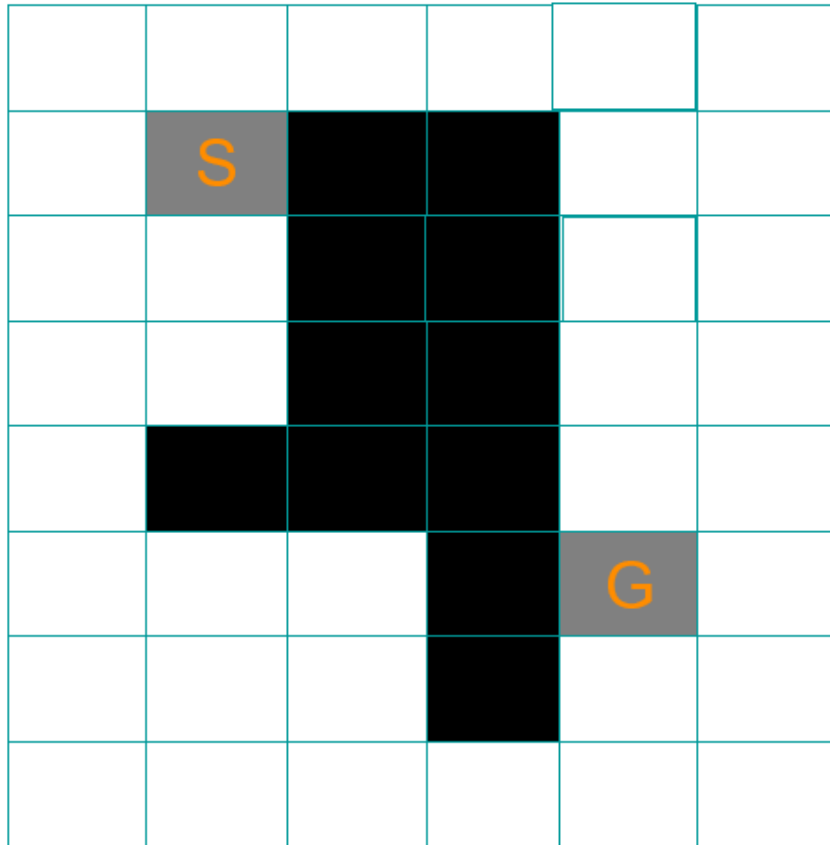


Wavefront Algorithm



What assumption is being made about the graph edges here?

S— start, G--goal



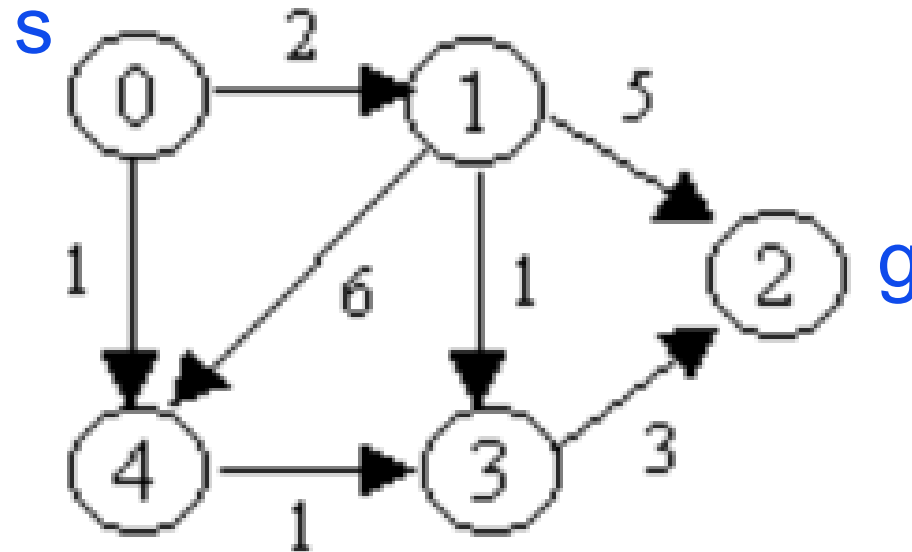
Get the path (reverse order)



A General Shortest Path Problem



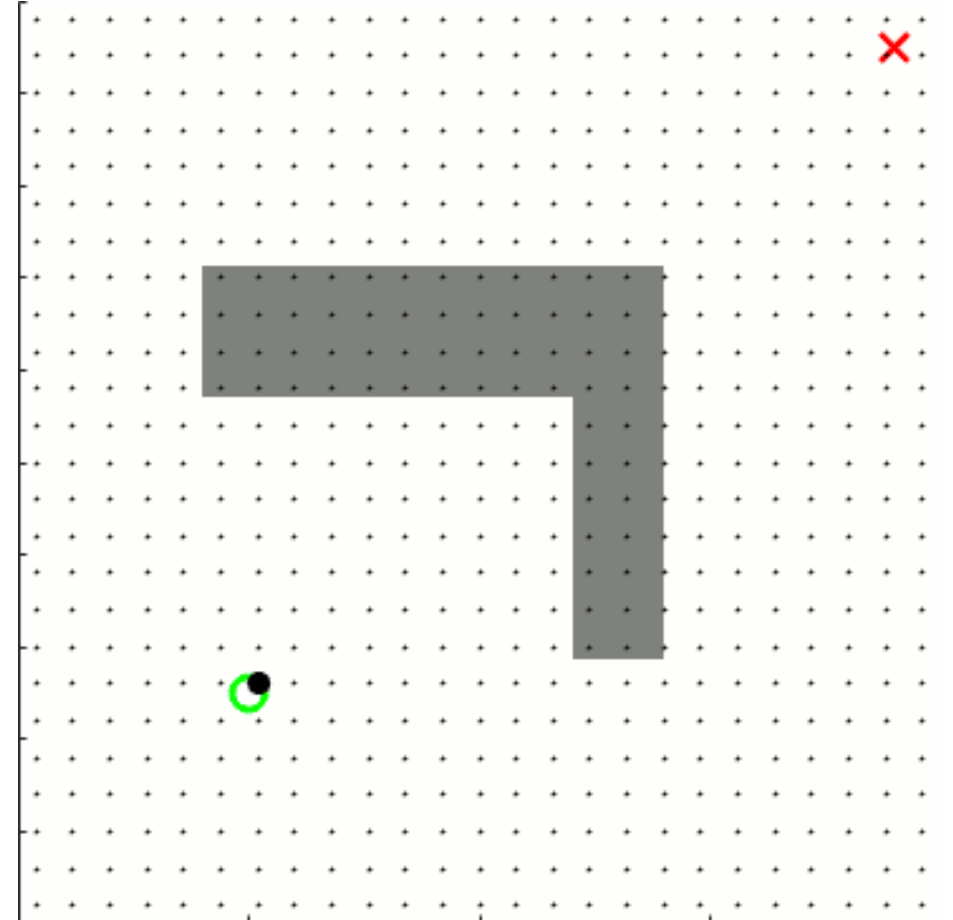
- Given a weighted graph and two vertices s and g
Find a shortest path from s to g (i.e. path with least sum of edge costs)



Dijkstra's Algorithm



- An algorithm that solves the shortest path problem
- Traverse the graph by visiting the next vertex that has the lowest path cost
- Maintain two sets:
 - Visited set (red)
 - Unvisited set (blue)

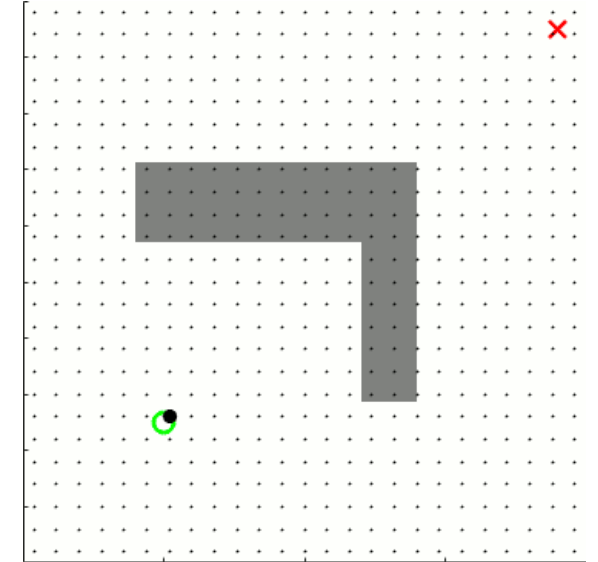


Dijkstra's Algorithm



Each node has a “path cost” and a “parent pointer”

1. Empty all parent pointers and path costs
2. Create empty *unvisited* and *visited* sets
3. Add start node to *unvisited* set with cost 0
4. Select the node in the *unvisited* set with minimum path cost
5. Move this selected node to the *visited* set
6. For each neighbour of the selected node:
 - If in *visited* set: do nothing
 - If in *unvisited* set: update the cost and parent if this path has lower cost
 - Otherwise: add it to the *unvisited* set with appropriate cost and parent
7. Repeat from step 4 until the *unvisited* set is empty or the goal is found



A* (“A star”) Algorithm



- An algorithm for the general shortest path problem that is faster than Dijkstra's algorithm in typical scenarios
- Relies on a user-defined heuristic estimate
- One change to the algorithm: Define the node costs f , g , and h as:

$$f(x) = g(x) + h(x)$$

Estimated path cost from start node to x to goal →

→ *Path-cost: cost of the shortest path from start node to x*

→ *Heuristic: Estimate of the distance to the goal*

- Heuristic must be admissible (not overestimate) and consistent (monotone increasing)
- Equivalent to Dijkstra's algorithm when $h(x) = 0$

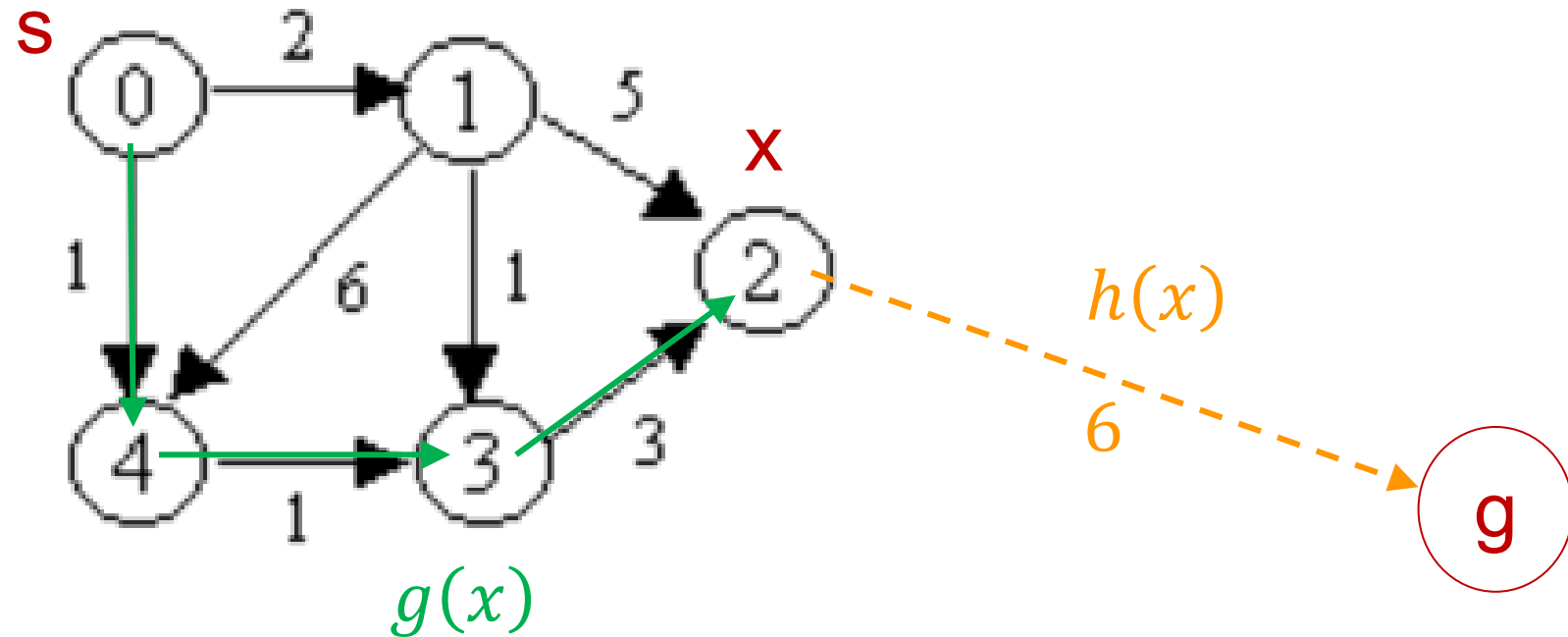


A* cost heuristic

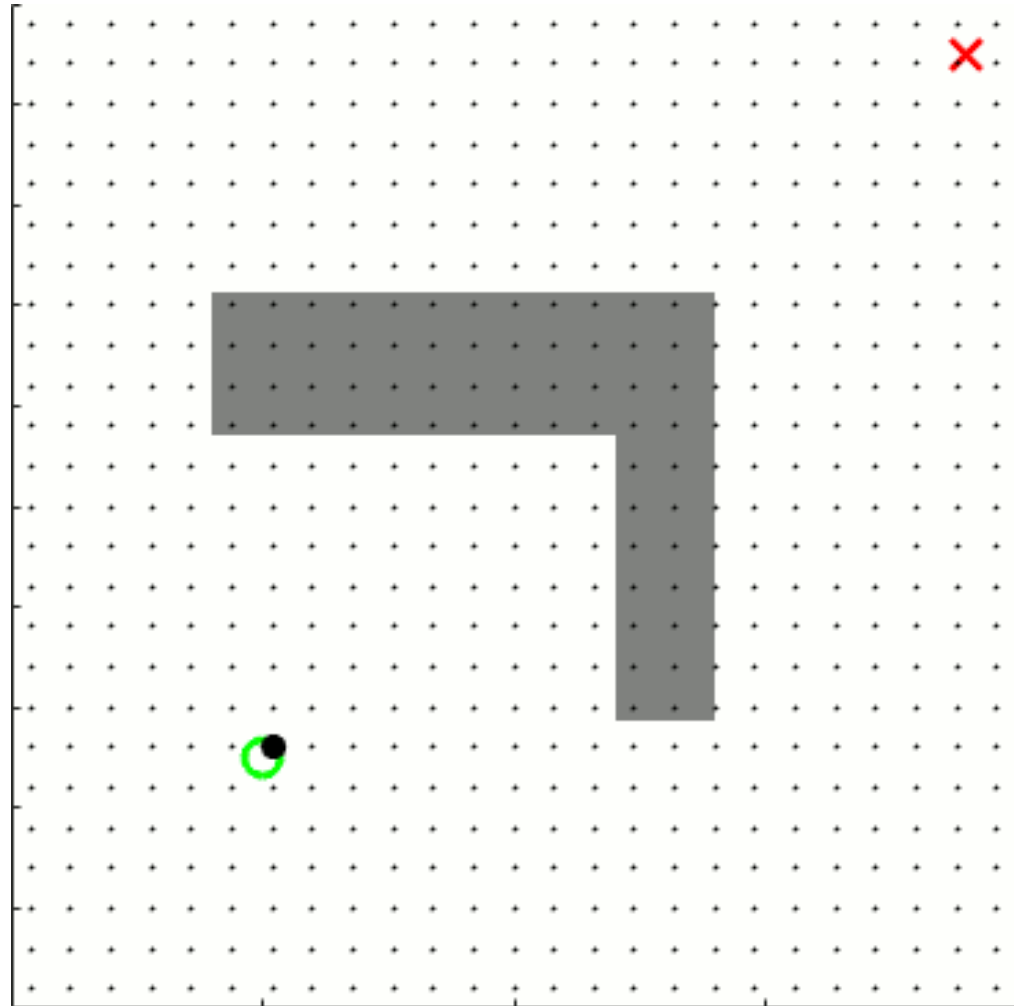
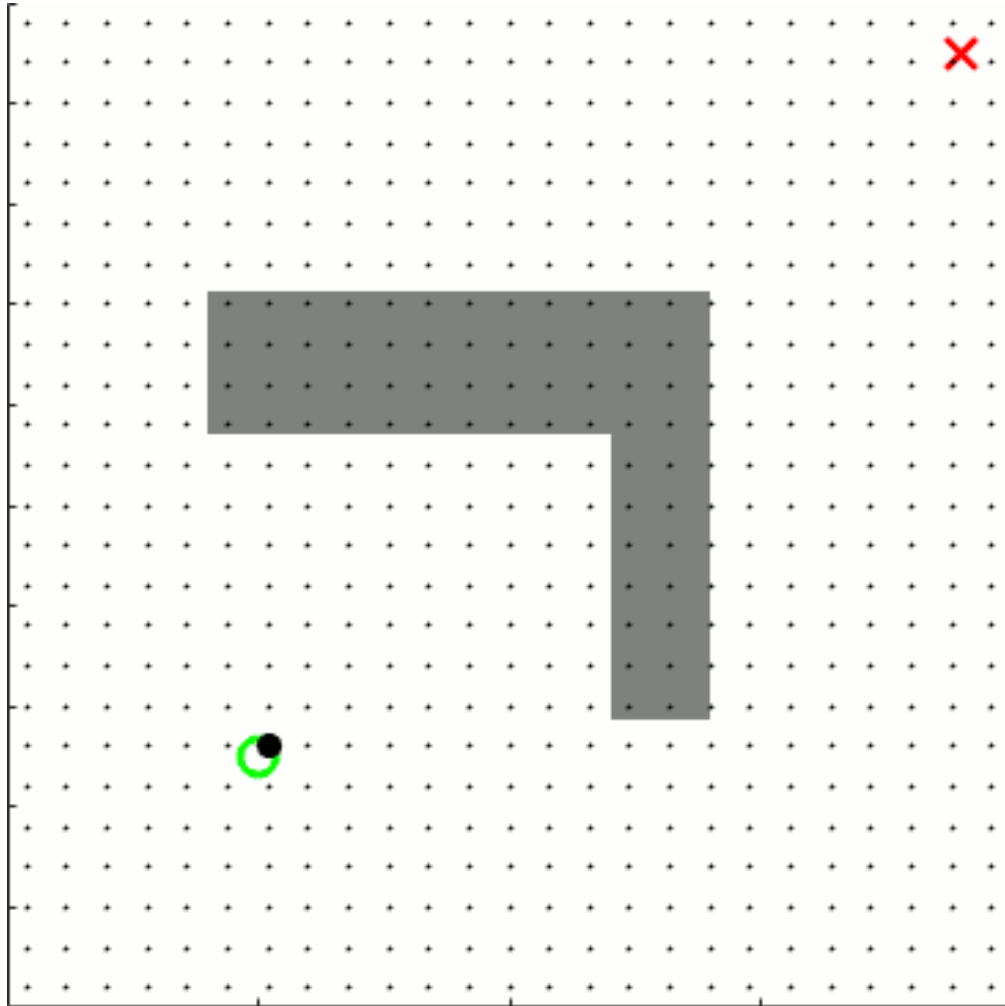
Estimated path cost from start node to x to goal $\rightarrow f(x) = g(x) + h(x)$

Path-cost: cost of the shortest path from start node to x

Heuristic: Estimate of the distance to the goal



Dijkstra's vs A*



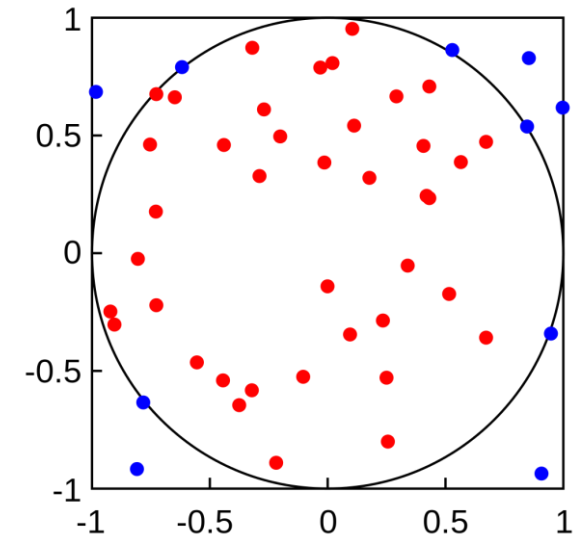


Sampling-Based Motion Planning

Randomised Algorithms (“Monte Carlo Algorithms”)



- General method for approaching computationally-difficult problems
- Replace parts of your algorithm that do an exhaustive search with random sampling
- Common approach in robotics:
 - Particle filter localisation
 - Probabilistic roadmaps (PRM) (**today**)
 - Randomly-exploring random trees (RRT) (**today**)
 - Monte Carlo tree search (MCTS) (**previous seminar**)

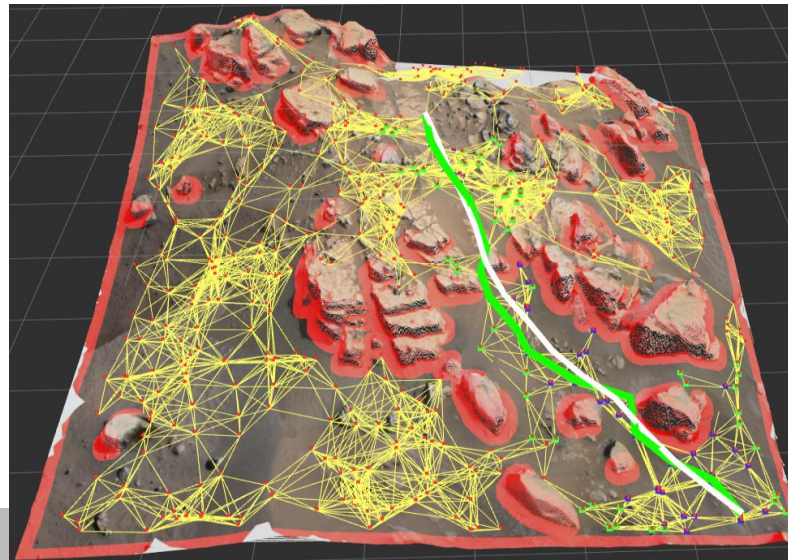


Probabilistic Roadmaps (PRM)



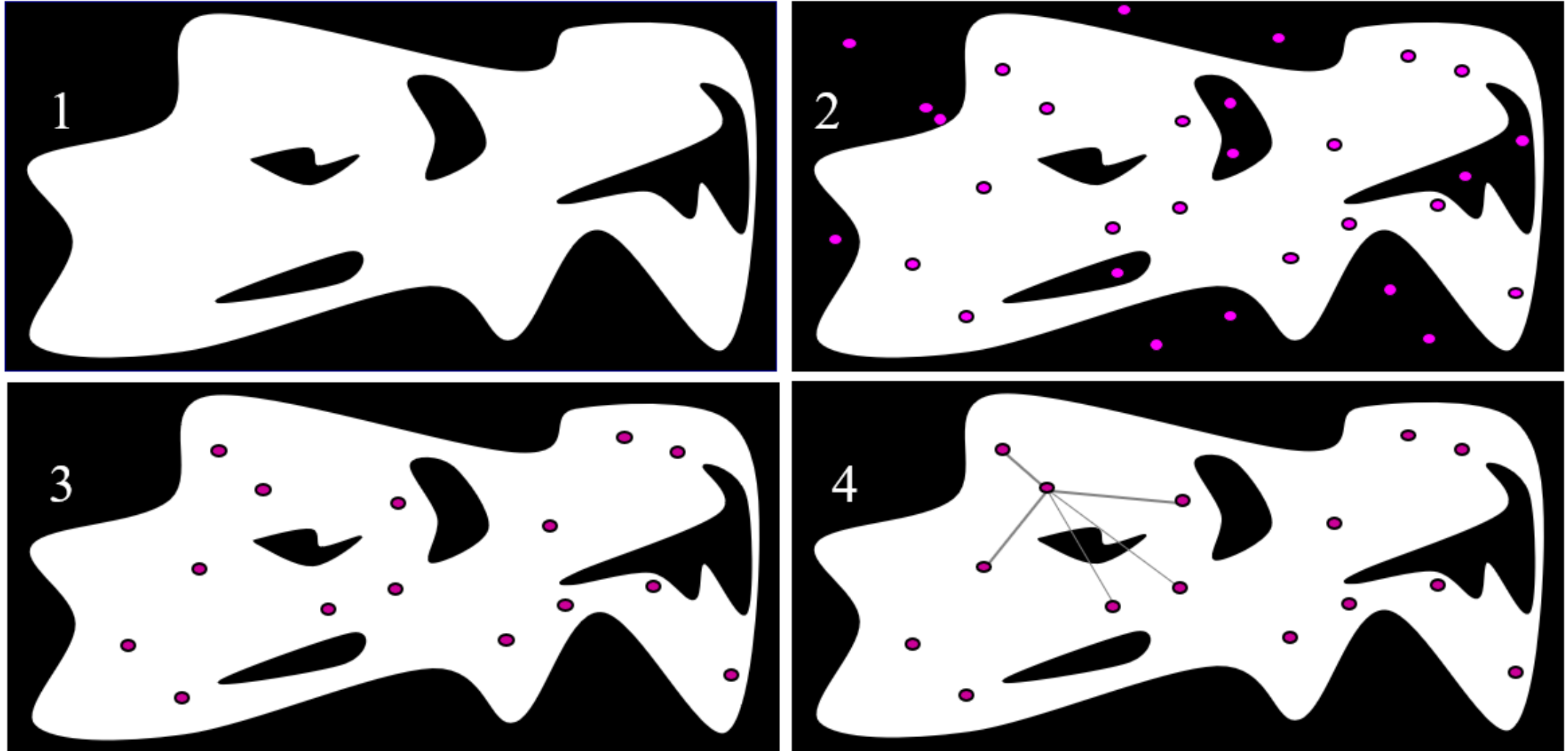
Generate random graphs using random sampling

1. Sample the C-space by generating a large number of samples
2. For every sample that is collision free, create a graph node at that location
3. Connect pairs of nodes that have a short, local path
4. After generating the graph, it can be searched with Dijkstra's or A*

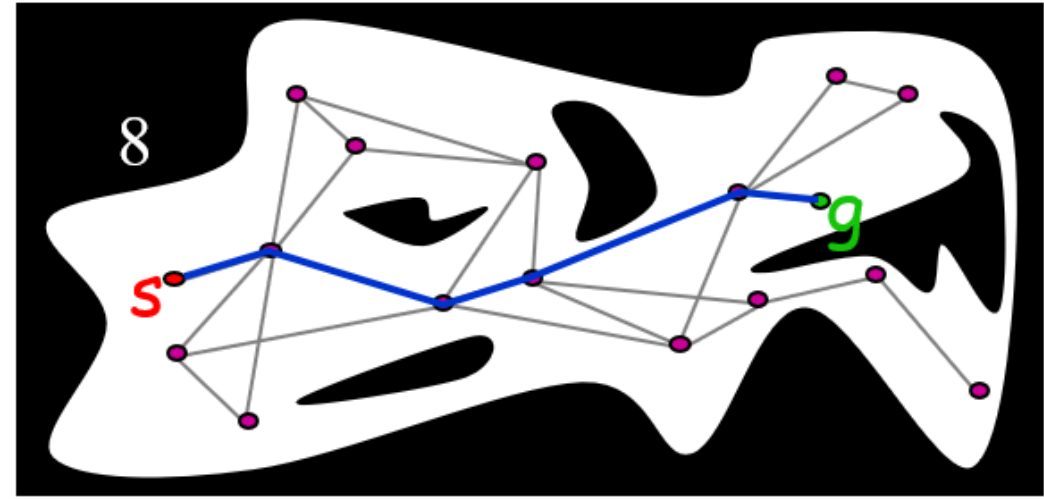
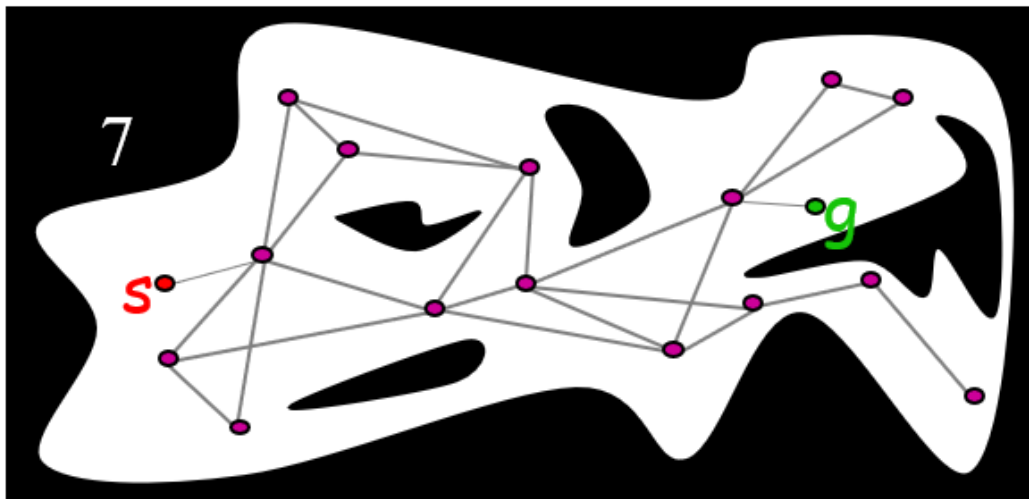
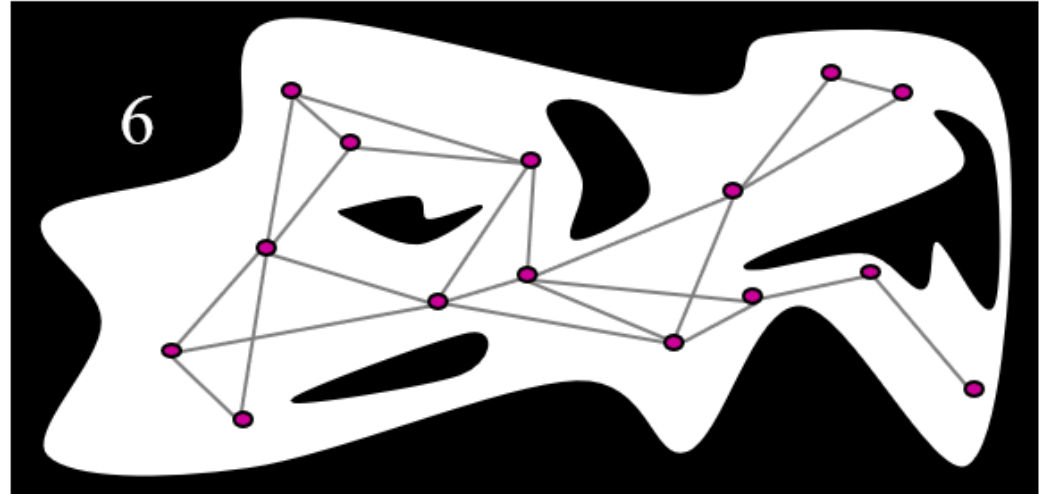
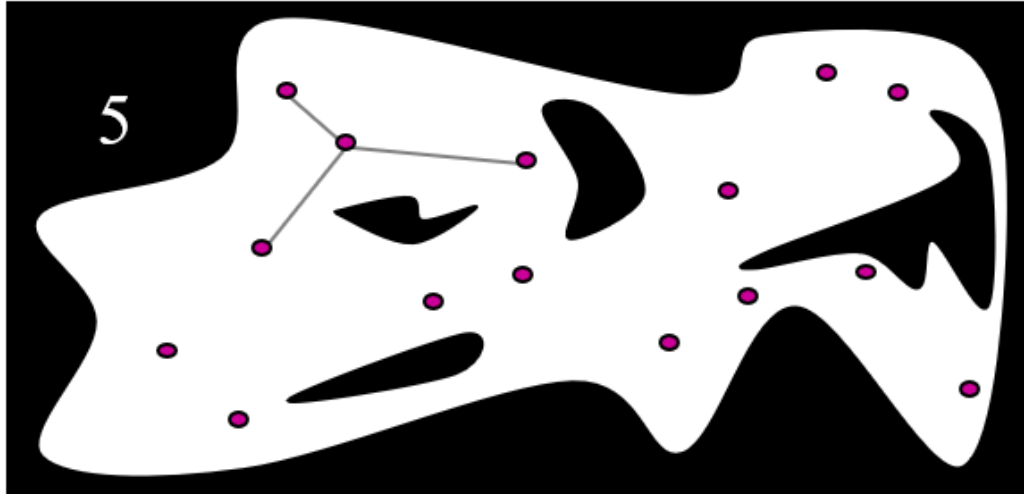


From 49274
Space Robotics

PRM Example



PRM Example



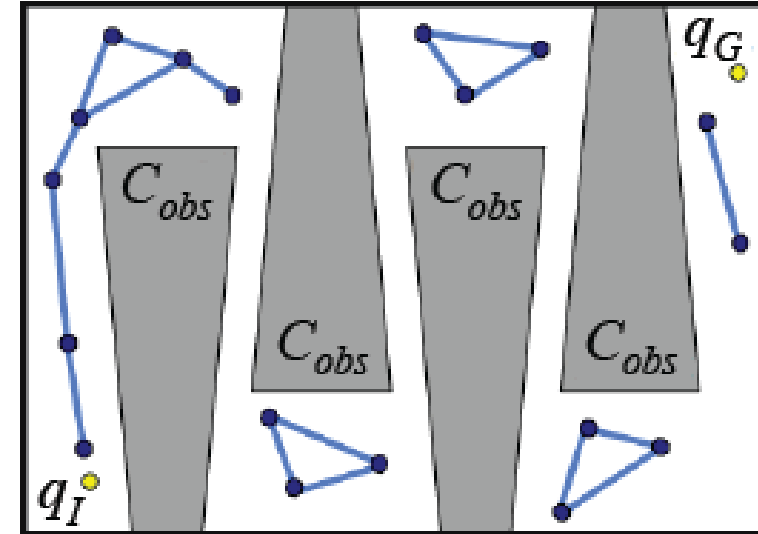
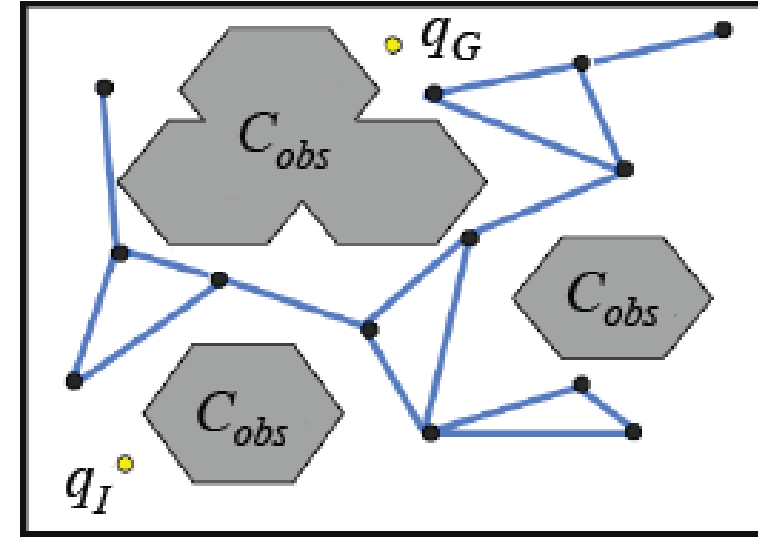
PRM

Advantages:

- Probabilistically complete
- Do not construct C-space
- Apply easily to high dimensional spaces
- Solved previously unsolved problems

Disadvantages:

- Does not work well at all problems
- Not optimal, not complete



RRT (Rapidly-Exploring Random Trees)

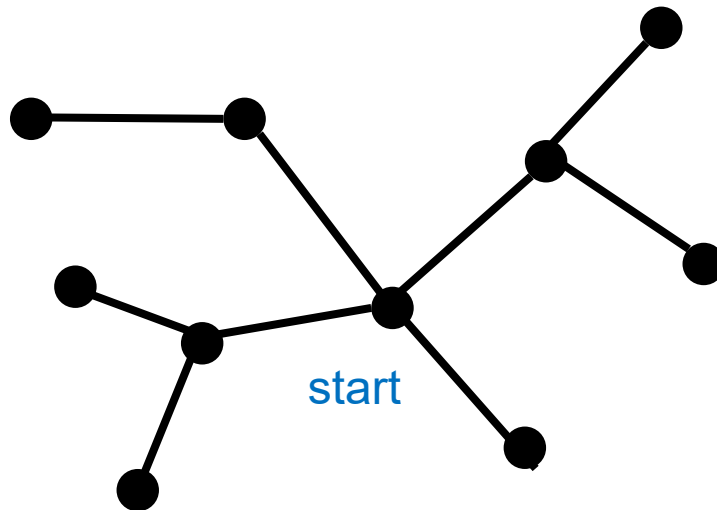


- A different type of randomised algorithm for path planning
- Conceptually simple: Incrementally grow a tree that explores the space
- Works well in high-dimensional state space
- Easily incorporate differential constraints
- Applicable to a broad class of problems
- Lots of variants, e.g. RRT* for asymptotic optimality

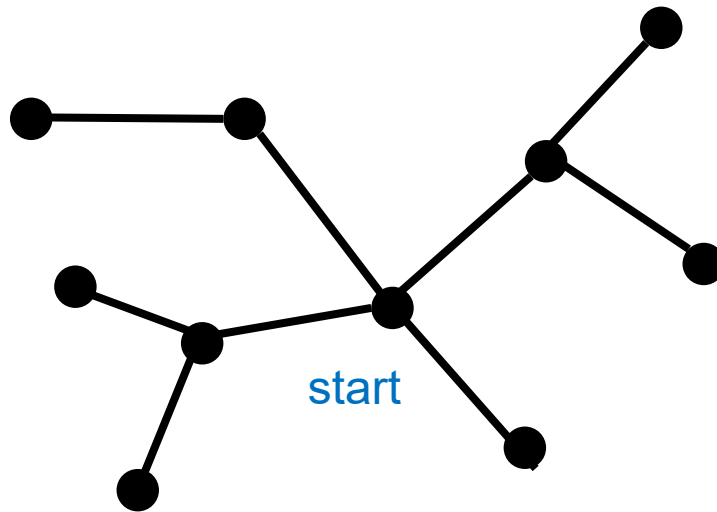
RRT Algorithm



tree after a few iterations



RRT Algorithm

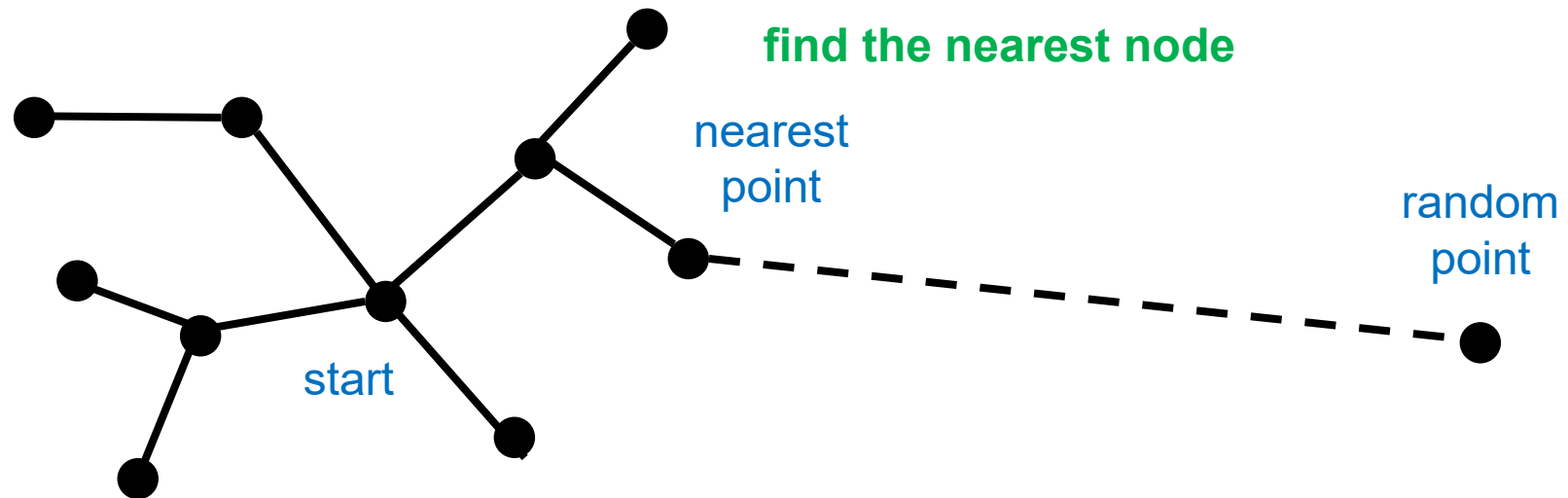


draw a random sample

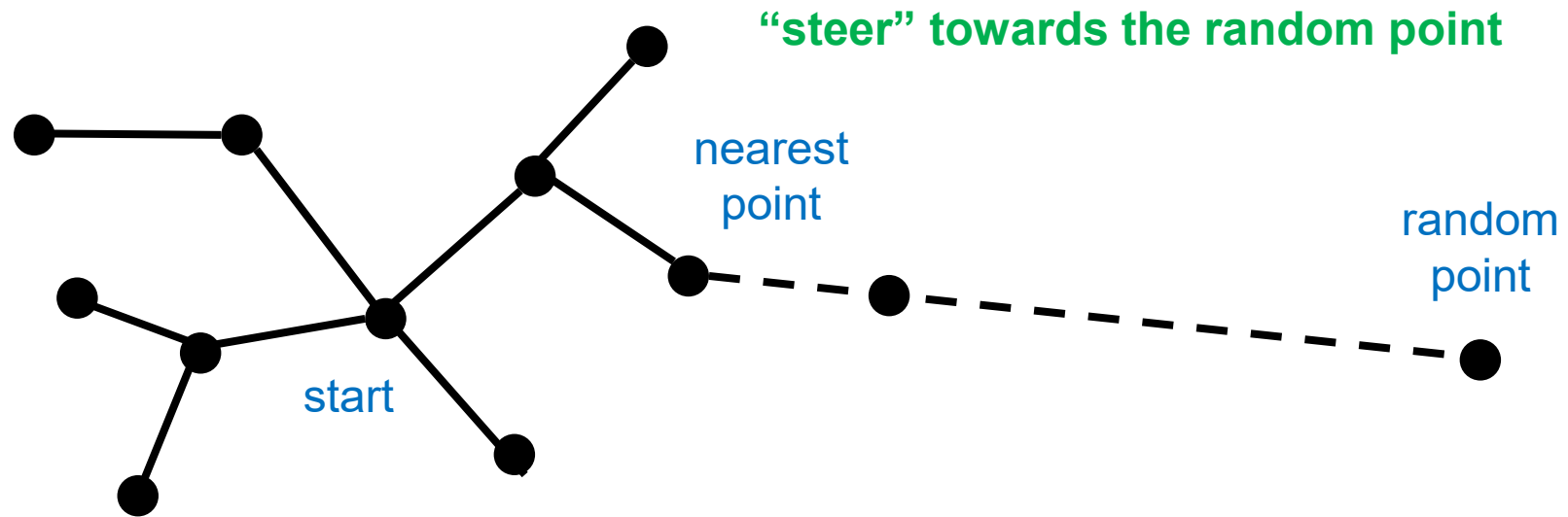
random
point



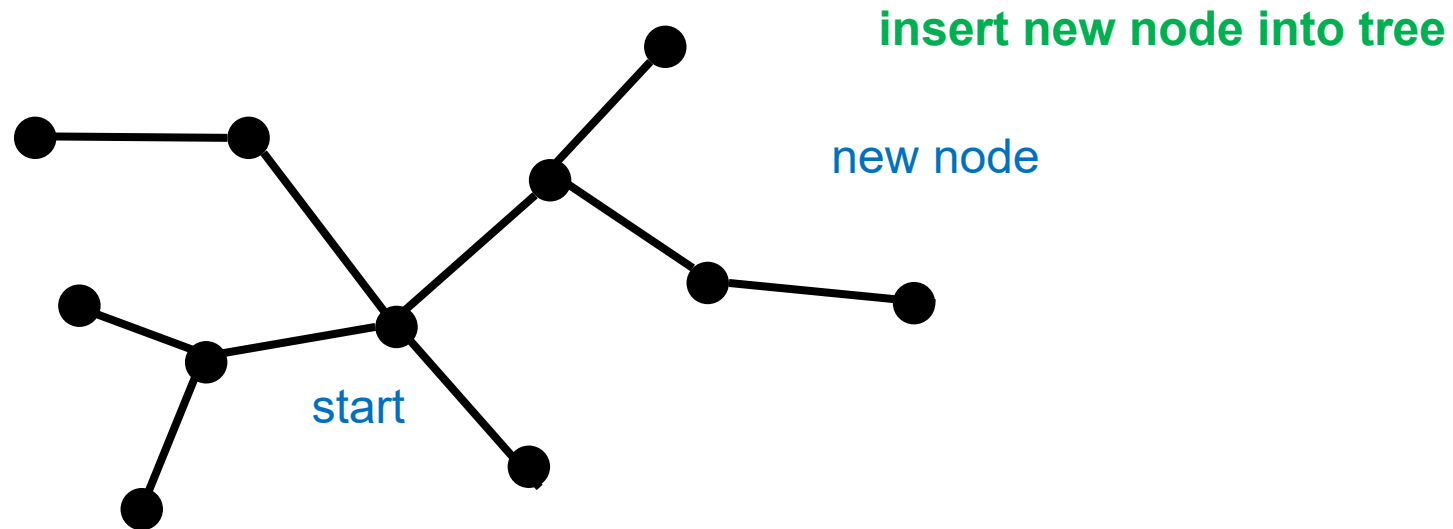
RRT Algorithm



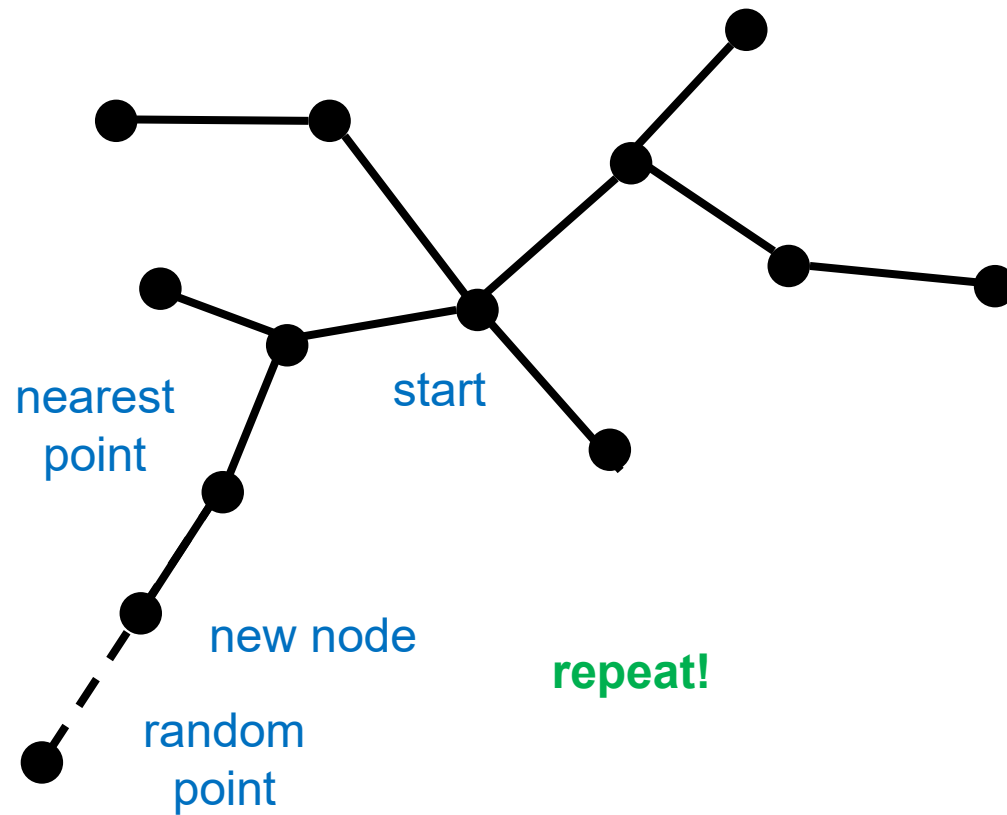
RRT Algorithm



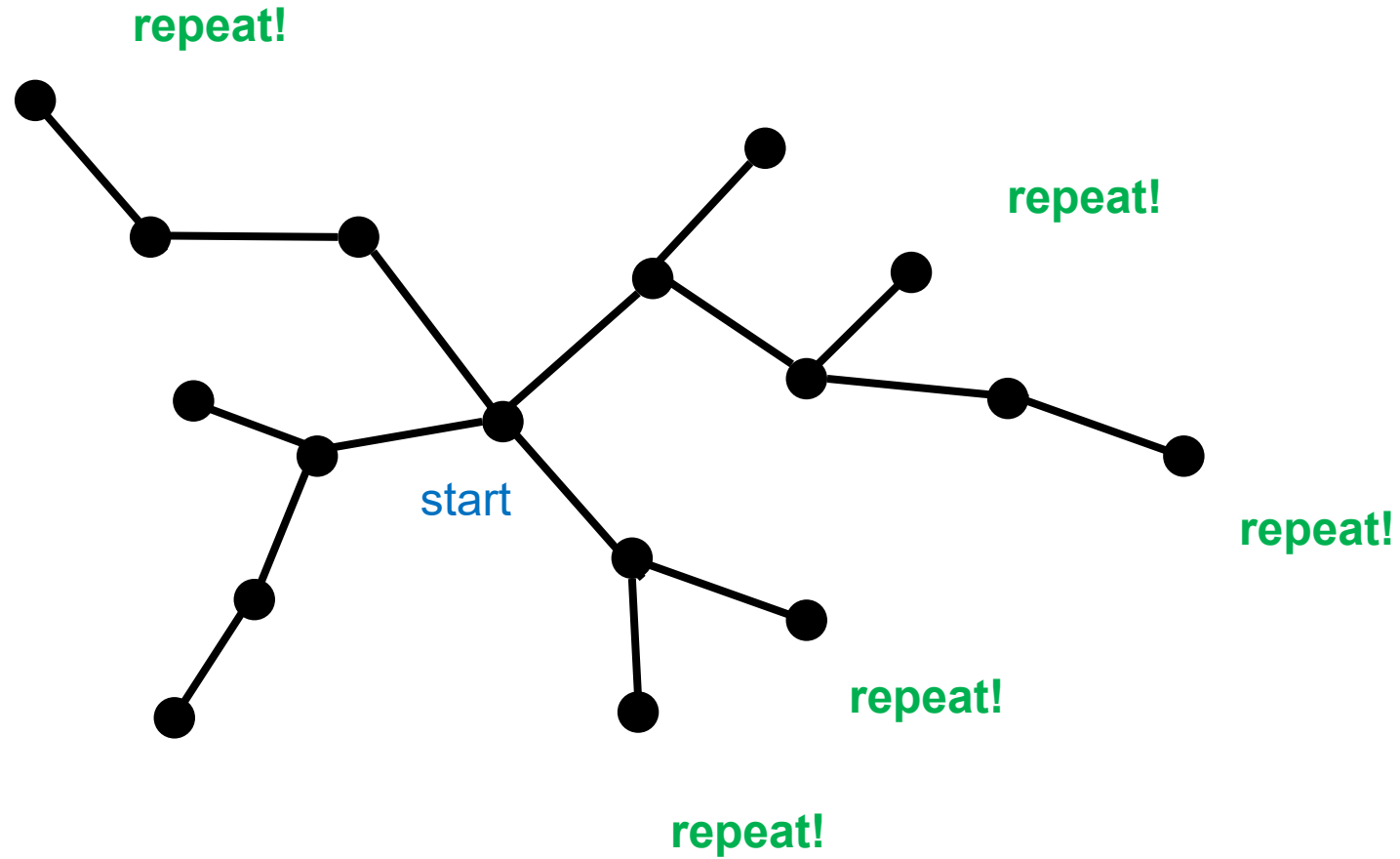
RRT Algorithm



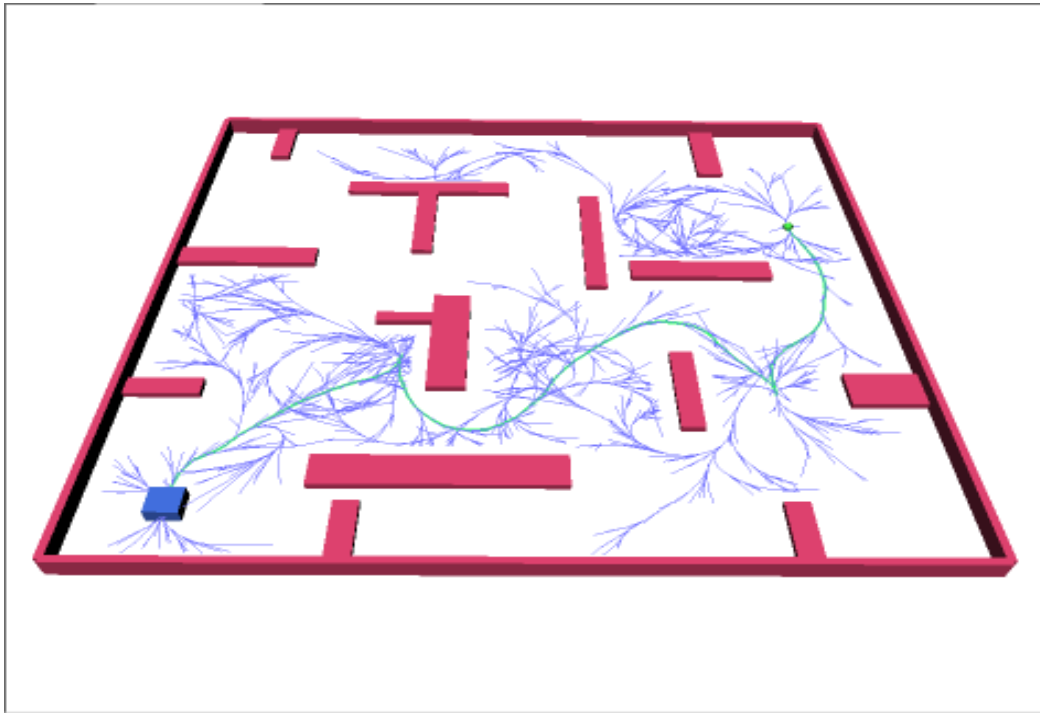
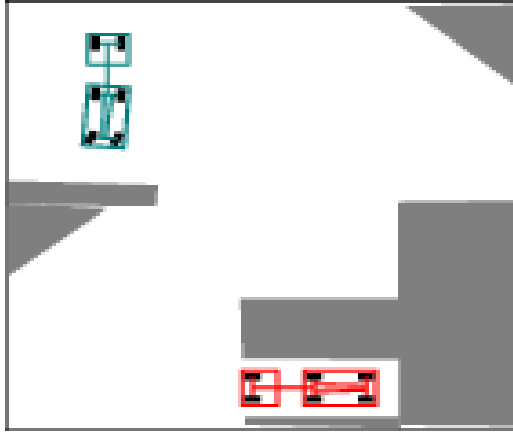
RRT Algorithm



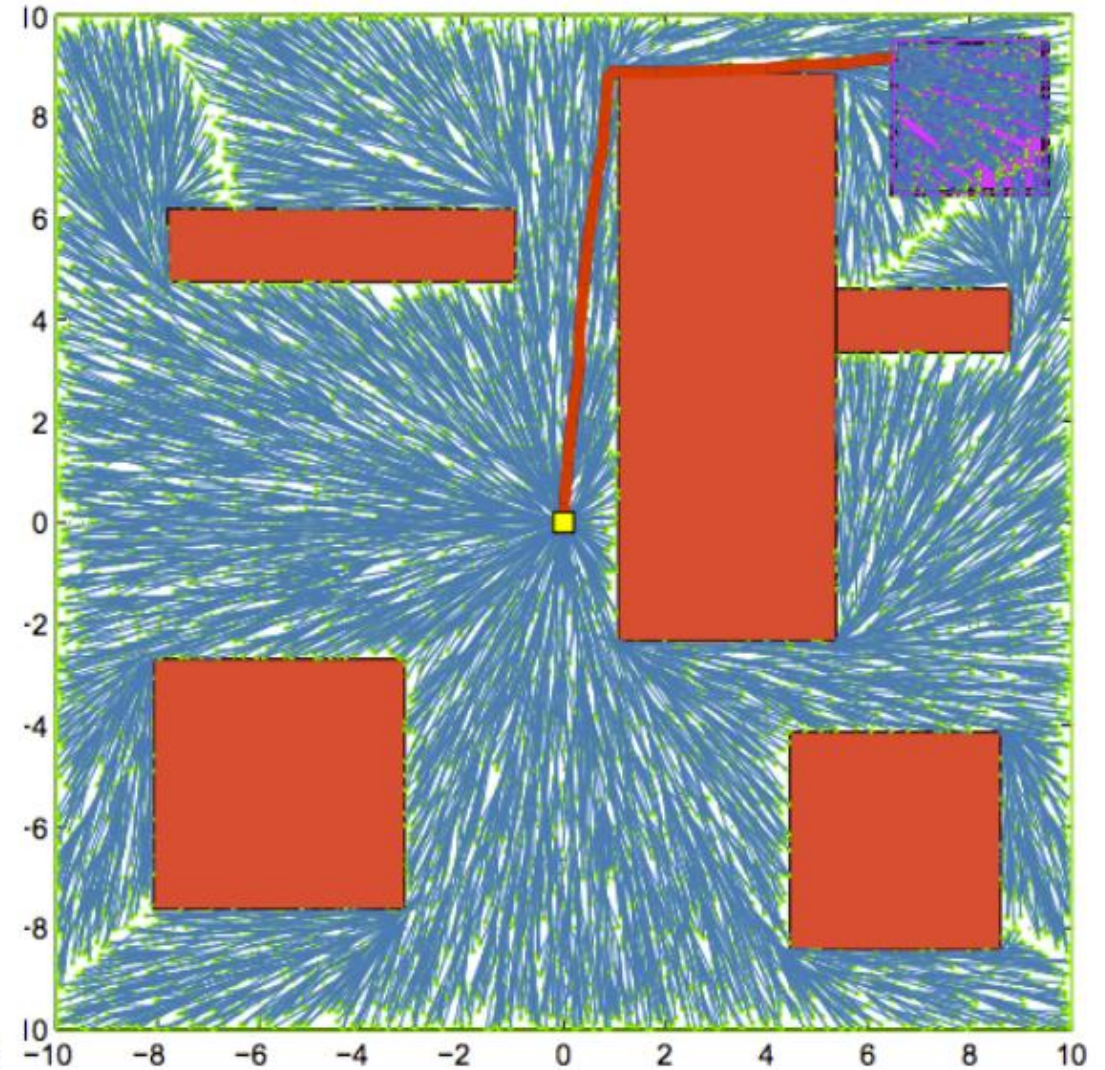
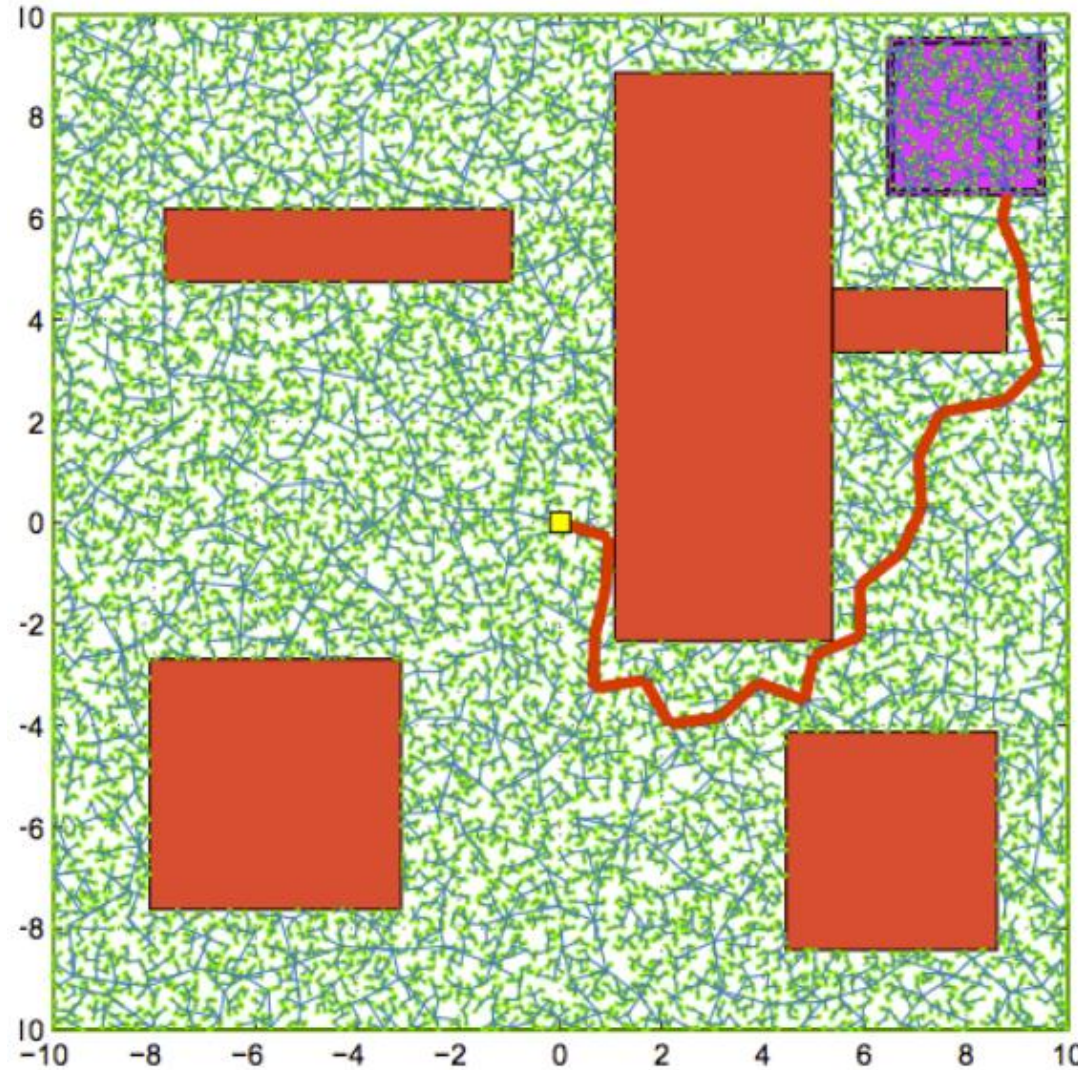
RRT Algorithm



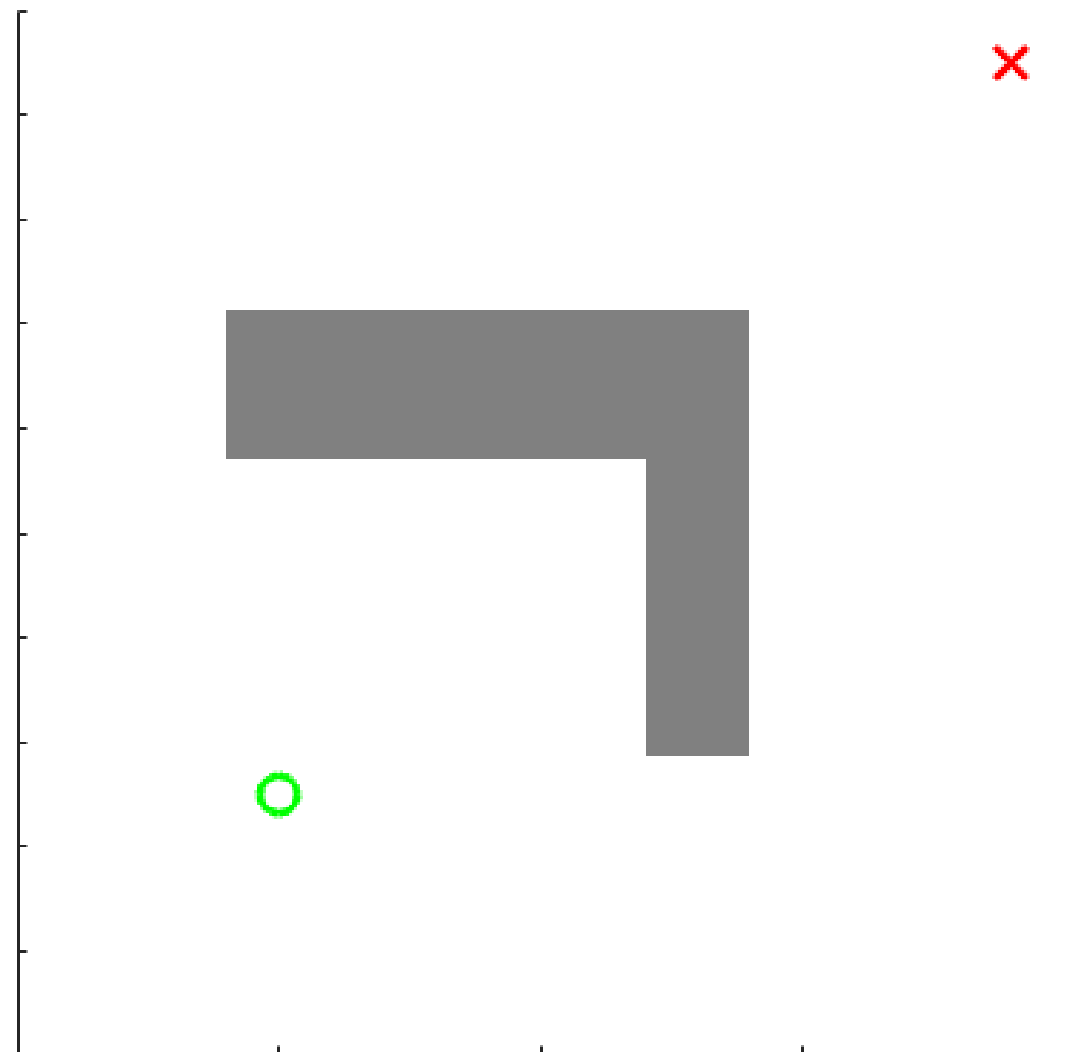
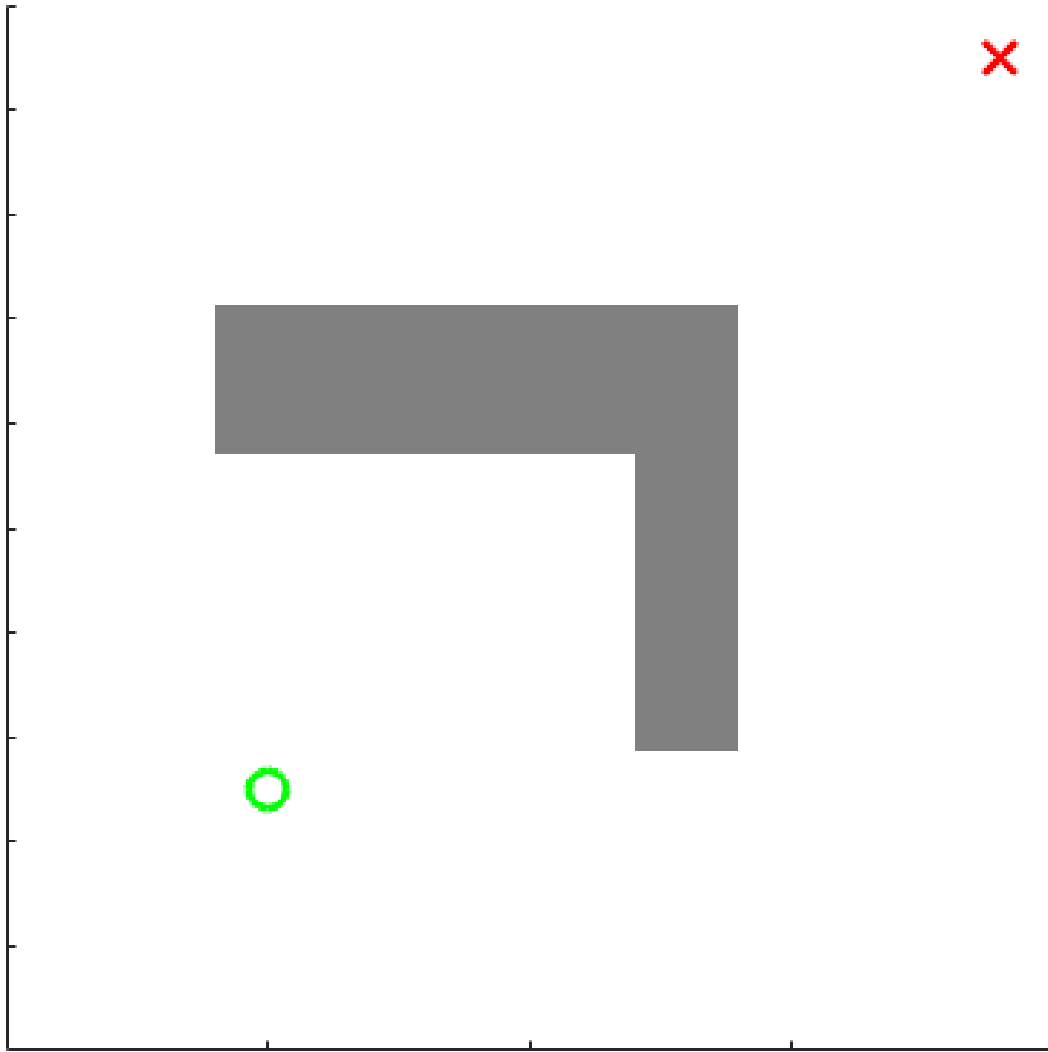
Examples



RRT vs RRT* (adds a “rewiring” step)



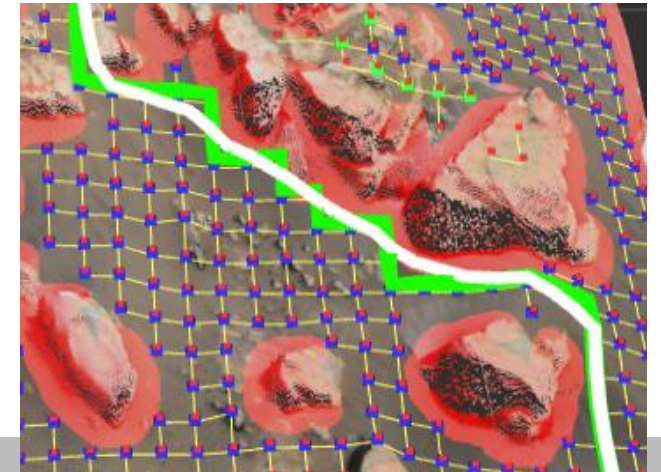
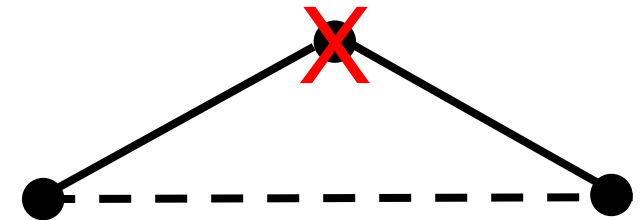
RRT vs PRM



Path Smoothing



- Randomised path planners tend to find paths that are not so great for execution:
 - Very jagged
 - Often longer than necessary
- In practice, smooth the path before using it
- Short-cutting:
 - Attempt to connect pairs of nodes along the path while skipping intermediate nodes
- Non-linear optimisation
 - Define a “smoothness” objective and optimise to update the path



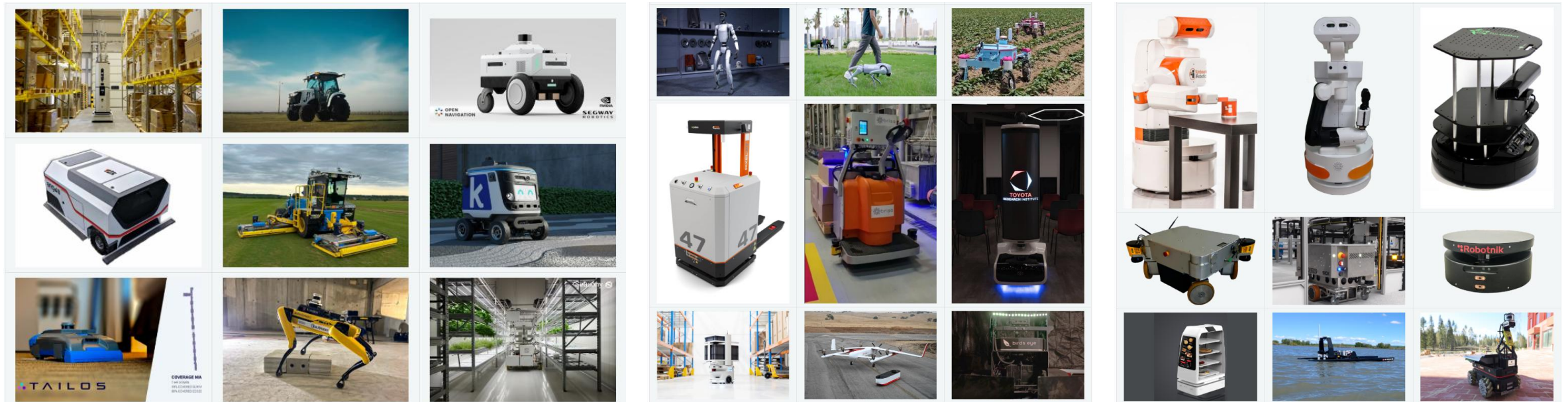


Nav2 Navigation Stack

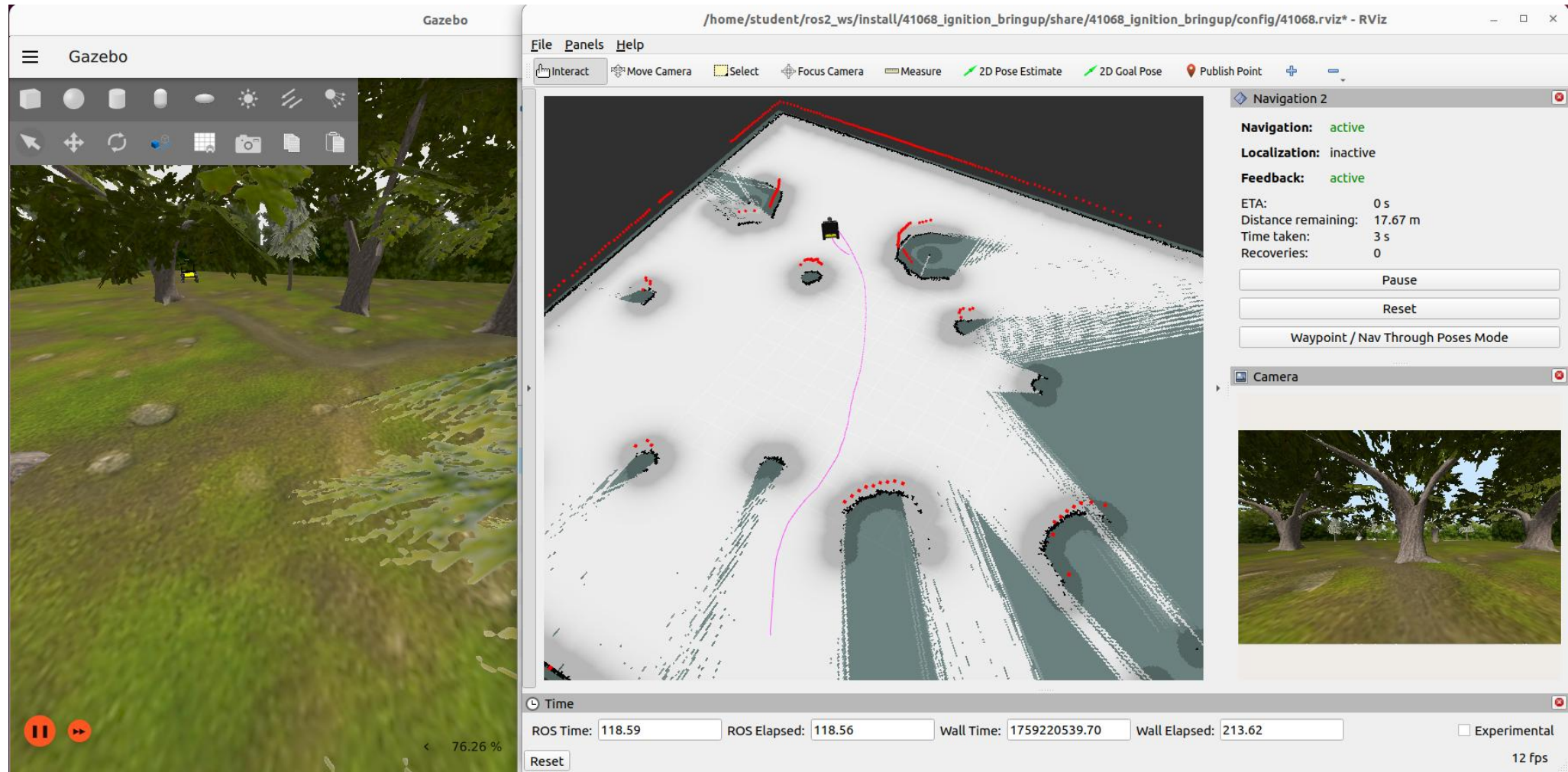
Nav2



- Default navigation tool for ROS2 – very extensively used across research and industry



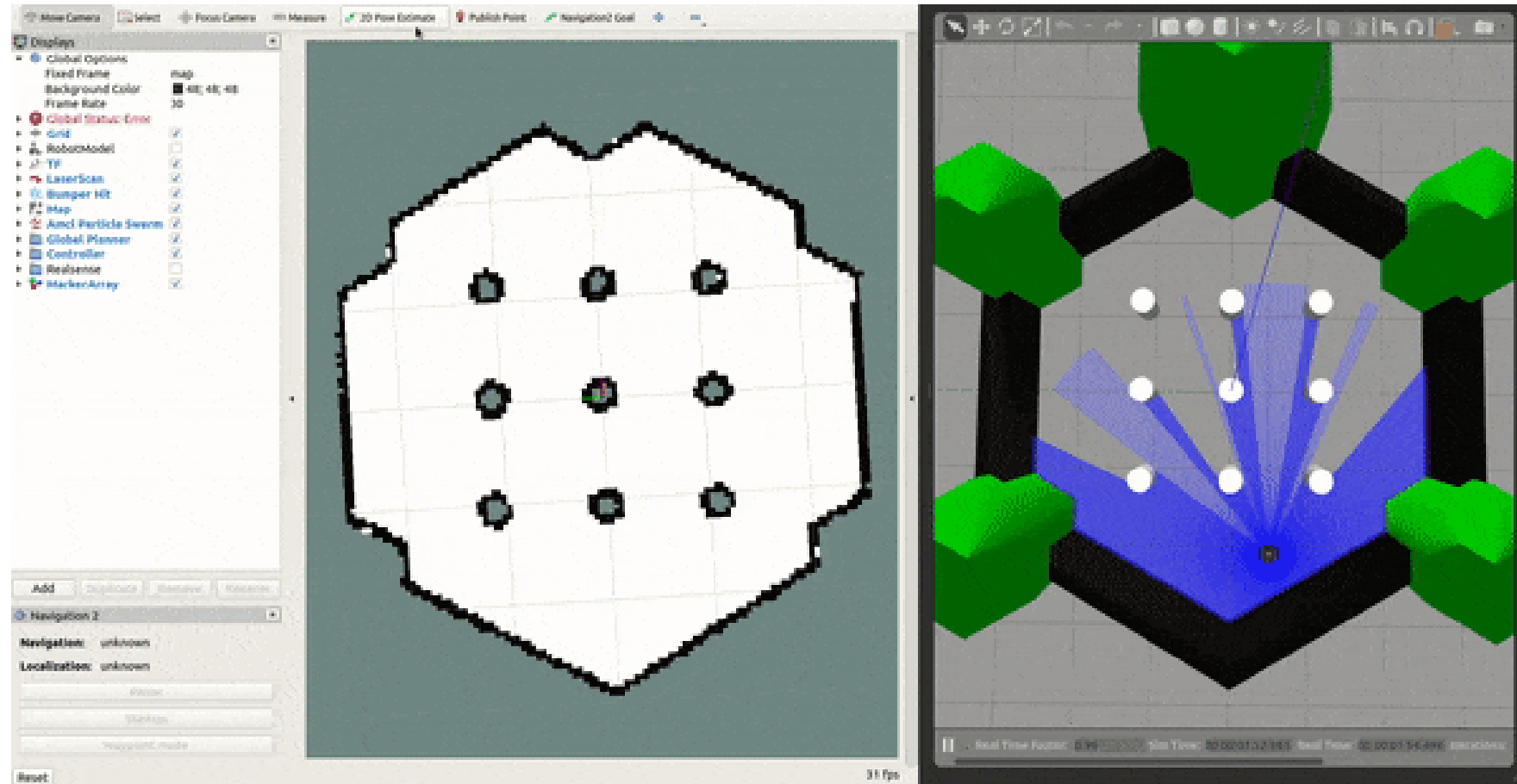
Example: 41068 package



Example: Turtlebot



➤ https://docs.nav2.org/getting_started/index.html



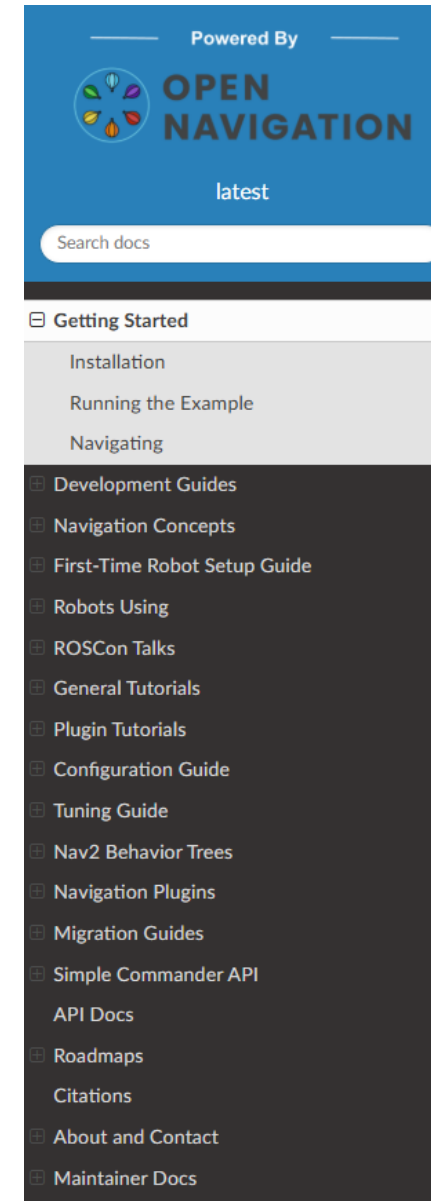
Nav2 Concept



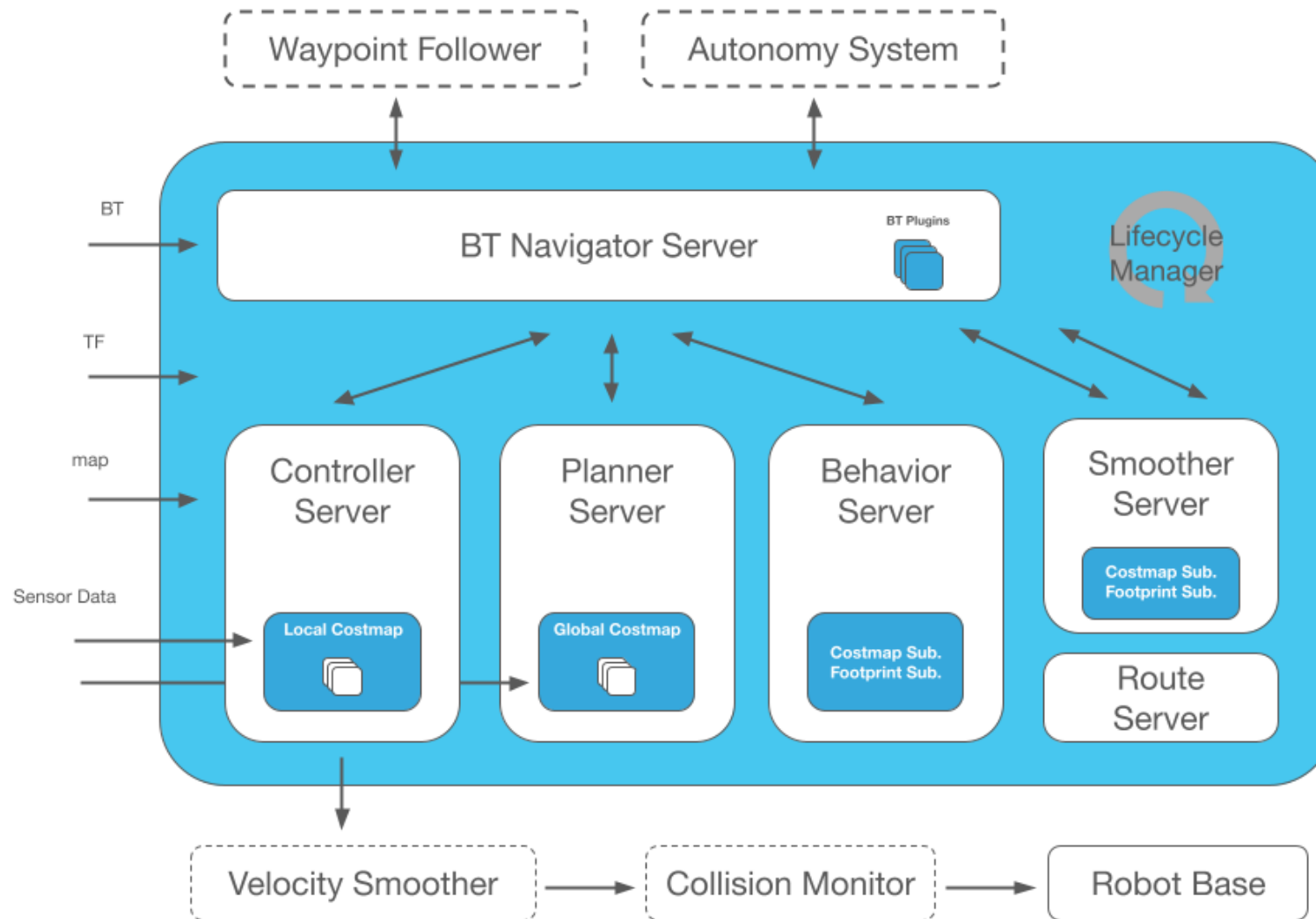
- Nav2 enables mobile robots to navigate through complex environments to complete user-defined application tasks with nearly any class of robot kinematics
- Move from Point A to Point B, sequences of points, object following, coverage, etc.
- Integrates perception, planning, control, localisation, visualisation
- Compute an environmental model from sensor and semantic data, dynamically path plan, compute velocities for motors, avoid obstacles, and structure higher-level robot behaviours
- Highly customisable through plugins and parameters
See config/nav_params.yaml in 41068 package

Documentation

- The documentation is very extensive, go read it!
<https://docs.nav2.org/>
- Overview from the developer:
<https://www.youtube.com/watch?v=QB7lOKp3ZDQ>
- Configuration guide:
<https://docs.nav2.org/configuration/index.html>
- Original academic paper:
<https://arxiv.org/pdf/2003.00368>



Overview



All of the plugins can be swapped and customised

Commander API



- From your code, you can interact with Nav2 using “actions”
 - Actions are like topics, but have a request and response, and goals and results:
<https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>
 - 41068 package provides example code
- The “Commander API” simplifies how to call and track the actions
Overview: https://docs.nav2.org/commander_api/index.html
- Code with examples:
https://github.com/ros-navigation/navigation2/tree/main/nav2_simple_commander

```
from nav2_simple_commander.robot_navigator import BasicNavigator
import rclpy

rclpy.init()
nav = BasicNavigator()

# ...

nav.setInitialPose(init_pose)
nav.waitForNav2Active() # if autostarted, else use LifecycleStartup()

# ...

path = nav.getPath(init_pose, goal_pose)
smoothed_path = nav.smoothPath(path)

# ...

nav.goToPose(goal_pose)
while not nav.isTaskComplete():
    feedback = nav.getFeedback()
    if feedback.navigation_duration > 600:
        nav.cancelTask()

# ...

result = nav.getResult()
if result == TaskResult.SUCCEEDED:
    print('Goal succeeded!')
elif result == TaskResult.CANCELED:
    print('Goal was canceled!')
elif result == TaskResult.FAILED:
    print('Goal failed!')
```



Reflection

Conclusions



- **Robot path planning** is the process of figuring out *how the robot moves from its current state to a target* given its map, constraints, uncertainties.
- **Artificial potential fields** is a simple path planning approach, but often gets stuck
- **Graph search methods**, including Dijkstra's and A*, plan a path globally over a graph representing the environment
- **Probabilistic Roadmap (PRM)** constructs a graph for A* using random sampling
- **Rapidly-Exploring Random Tree (RRT)** randomly expands a tree over the configuration space to find a path
- **Nav2** is the default ROS2 package for navigation

Questions for your 41068 Project



- Introduce standard path planning techniques
- Your project must have some decision making / path planning
- How will your robot navigate around the environment?
 - You are probably using the Nav2 stack
 - But how does this work?
 - How can you modify it?
 - Or implement it yourself?

